

# Advance Java All ReadMe Notes – Mrs. Madhura Anturkar

Version Java EE 8 (J2EE 1.8) maintained under Oracle / Jakarta EE 8 (maintained by eclipse foundation)

## 1. What is J2EE ?

Consists of specifications only .

### Which specs ? (Rules or contract )

Specifications of services required for any enterprise application.

### What is enterprise application ?

An enterprise application (EA) is a large software system platform designed to operate in a corporate environment .

It includes online shopping and payment processing, interactive product catalogs, computerized billing systems, security, content management, IT service management, business intelligence, human resource management, manufacturing, process automation, enterprise resource planning ....

### These specifications include ---

**Servlet** API, JSP (Java server page) API, Security, Connection pooling, EJB (Enterprise Java Bean), JNDI (Naming service -- Java naming & directory i/f), JPA (java persistence API), JMS (java messaging service), Java Mail, Java Server Faces, Java Transaction API, Webservices support (SOAP/REST) etc...

### Vendor of J2EE specs -- Oracle / Sun / Eclipse

Implementation -- left to vendors (J2EE server vendors)

J2EE compliant web server --- Apache -- Tomcat (web container)

Services implemented --- servlet API, JSP API, Security, Connection pooling, JNDI (naming service)

J2EE complaint application server --- web container + EJB (enterprise java bean) container

+ ALL J2EE services implementation

J2EE server Vendors & Products

Apache -- tomcat (web server) / Tomee (app server)

Oracle / Sun --- reference implementation --- Glassfish

Red Hat -- JBoss (wild fly)

Oracle / BEA -- weblogic

IBM -- Websphere

## 2. WHY J2EE

1. Can support different types of clnts --- thin client (web clnt)

thick clnt --- application clnt (eg : TCP client)

smart clnts -- mobile clnts

2. J2EE server independence --- Create & deploy server side appln -- on ANY J2ee compliant server --- guaranteed to produce SAME results w/o touching or re-deploying on ANY other J2EE server

3. Ready made implementation of primary services (eg --- security, conn, pooling, email....) --- so that J2EE developer DOESN'T have to worry about primary services --- rather can concentrate on actual business logic.

3. Layers involved in HTTP request-response flow (refer to day1-data\day1\_help\diags\request-response-flow.png)

Web browser sends the request (URL)

eg : http://www.abc.com:8080/day1.1

/day1.1 --- root / context path /web app name

Host --Web server--Web Container(server side JVM)--Web application---HTML/JSP/Servlet....

4. What is dyn web application --- server side appln --deployed on web server --- meant for servicing typically web clnts(thin) -- using application layer protocol HTTP /HTTPS

(ref : diag request-resp flow)

Read --HTTP basics including request & response structure from day1-data\day1\_help\j2ee\_prerequisites\HTTP Basics

5. Objective ? : Creating & deploying dyn web appln on Tomcat -- For HTML content

6. IDE automatically creates J2EE compliant web application folder structure .

Its details -- Refer to diag (J2EE compliant web app folder structure)

7. What is Web container --- (WC) & its jobs

1. Server side JVM residing within web server.

Its run-time env for dyn web components(Servlet & JSP,Filter) .

Jobs ---

1. Creating Http Request & Http response objects

2. Controlling life-cycle of dyn web comps (manages life cycle of servlet,JSP,Filters)

3. Giving ready-made support for services --- Naming,security,Conn pooling .

4. Handling concurrent request from multiple clients .

5. Managing session tracking...

8. What is web.xml --- Deployment descriptor one per web appln

created by -- dev

who reads it -- WC

when --- @ deployment

what --- dep instrs --- welcome page, servlet deployment tags, sess config, sec config.....

9. Why servlets? --- To add dynamic nature to the web application

### What is a servlet ?

-- Java class (with NO main method) -- represents dynamic web comp - whose life cycle will be managed by WC(web container : server side JVM)

no main method

life cycle methods --- init,service,destroy

Job list

1. Request processing

2. B.L

3. Dynamic response generation

4. Data access logic(DAO class --managing DAO layer)

5. Page navigation

Servlet API details --refer to diag servlet-api.png

Objective 1: Test basic servlet life cycle -- init , service ,destroy

10. Creating & deploying Hello Servlet.

Deployment of the servlet

### 1. Via annotation

eg : @WebServlet(value="/validate")

public class LoginServlet extends H.S {...}

Map :

key -- /validate

value -- F.Q servlet cls name

URL : http://host:port/day1.1/validate?....

2. Using XML tags

How to deploy a servlet w/o annotations --- via XML tags

web.xml

<servlet>

<servlet-name>abc</servlet-name>

<servlet-class>pages.SecondServlet</servlet-class>

</servlet>

<servlet-mapping>

<servlet-name>abc</servlet-name>

<url-pattern>/test2</url-pattern>

</servlet-mapping>

WC : map

key : /test2

value : pages.SecondServlet

eg URL --http://host:port/day1\_web/hello

At the time of web app deployment ---WC tries to populate map of url patterns , from XML tags (from web.xml).

Later ---it will check for @WebServlet annotation

Objective 2: Test basic servlet life cycle -- init , service ,destroy (deployed via xml)

**How to read request params sent from the clnt ?**

javax.servlet.ServletException i/f methods

1. public String getParameter(String paramName)
2. public String[] getParameterValues(String paramName)

Objective 3 : Accept different type of i/ps from user , in HTML form. Write a servlet to display request parameters.

=====

## Why HttpServlet class is declared as abstract class BUT with 100 % concrete functionality ?

It is abstract because the implementations of key servicing methods have to be provided by (e.g. overridden by) servlet developer. Since it's abstract , it's instance can't be created.

A subclass of HttpServlet must override at least one method, usually one of these:

doGet, if the servlet supports HTTP GET requests

doPost, for HTTP POST requests

doPut, for HTTP PUT requests

doDelete, for HTTP DELETE requests

init and destroy, to manage resources that are held for the life of the servlet

If you extend the class without overriding any methods, you will get a useless servlet; i.e. it will give an error response for all requests.(HTTP 405 : Method not implemented) . So , if the class was not abstract, then any direct instance of HttpServlet would be useless.

So the reason for making the HttpServlet class abstract is to prevent a programming error.

As a servlet developer , you can choose to override the functionality of your requirement (eg : doPost)

& ignore other methods.

=====

## Page Navigation Techniques

Page Navigation=Taking user from 1 page to another page.

### 2 Ways

#### 1. Client Pull

Taking the client to the next page in the NEXT request

1.1 User takes some action --eg : clicking on a button or link & then client browser generates new URL to take user to the next page.

#### 1.2 Redirect Scenario

User doesn't take any action. Client browser automatically generates new URL to take user to the next page.(next page can be from same web appln , or diff web appln on same server or any web page on any svr)

API of HttpServletResponse

public void sendRedirect(String redirectURL)

eg : For redirecting client from Servlet1 (/s1) to Servlet2 (/s2) , use

response.sendRedirect("/s2");

## 2. Server Pull.

Taking the client to the next page in the same request.

Also known as resource chaining or request dispatching technique.

Client sends the request to the servlet / JSP. Same request can be chained to the next page for further servicing of the request.

### Steps

**1. Create Request Dispatcher** object for wrapping the next page(resource --can be static or dynamic)

API of ServletRequest

```
javax.servlet.RequestDispatcher getRequestDispatcher(String path)
```

### 2. Forward scenario

API of RequestDispatcher

```
public void forward(ServletRequest rq, ServletResponse rs)
```

This method allows one servlet to do initial processing of a request and another resource to generate the response. (i.e division of responsibility)

Uncommitted output in the response buffer is automatically cleared before the forward.

If the response already has been committed(pw flushed or closed) , this method throws an `IllegalStateException`.

Limitation --only last page in the chain can generate dynamic response.

### 3. Include scenario

API of RequestDispatcher

```
public void include(ServletRequest rq, ServletResponse rs)
```

Includes the content of a resource @run time (servlet, JSP page, HTML file) in the response. -- server-side includes.

Limitation -- The included servlet/JSP cannot change the response status code or set headers; any attempt to make a change is ignored.

=====

## What is a Session?

Session is a conversational state between client and server and it can consists of multiple request and response between client and server. Since HTTP and Web Server both are stateless, the only way to maintain a session is when some unique information about the session is passed between server and client in every request and response.

HTTP protocol and Web Servers are stateless, what it means is that for web server every request is a new request to process and they cant identify if its coming from client that has been sending request previously.

But sometimes in web applications, we should know who the client is and process the request accordingly. For example, a shopping cart application should know who is sending the request to add an item and in which cart the item has to be added or who is sending checkout request so that it can charge the amount to correct client.

## What is the need of session tracking?

1. To identify the clnt among multiple clnts

2. To remember the conversational state of the clnt(eg : list of the purchased books/ shopping cart) throughout current sessio

session = Represents duration or time interval

Consists of all requests/resps coming from/ sent to SAME clnt from login to logout or till session expiration tmout.

There are several techniques for session tracking.

1. Plain Cookie based scenario
2. HttpSession interface
3. HttpSession + URL rewriting

---

## Techniques

### 1. Plain Cookie based scenario

#### What is a cookie?

Cookie is small amount of text data.

Created by -- server (servlet or JSP prog or WC) & downloaded (sent) to clnt browser---within response header

Cookie represents data shared across multiple dyn pages from the SAME web appln.(meant for the same client)

Steps :

1. Create cookie/s instance/s

```
javax.servlet.http.Cookie(String cName,String cVal)
```

- 2.Add the cookie/s to the resp hdr.

HttpServletResponse API :

```
void addCookie(Cookie c)
```

3. To retrieve the cookies :

HttpServletRequest :

```
Cookie[] getCookies()
```

- 4.Cookie class methods :

```
String getName()
```

```
String getValue()
```

```
void setMaxAge(int ageInSeconds)
```

def age =-1 ---> browser stores cookie in cache

=0 ---> clnt browser should delete cookie

>0 --- persistent cookie --to be stored on clnt's hard disk.

```
int getMaxAge()
```

#### Disadvantages of pure cookie based scenario

0. Web developer (servlet prog) has to manage cookies.

1. Cookies can handle only text data : storing Java obj or bin data difficult.

2. As no of cookies inc., it will result into increased net traffic.

3. In cookie based approach : entire state of the clnt is saved on the clnt side. If the clnt browser rejects the cookies: state will be lost : session tracking fails.

How to redirect client automatically to next page ? (in the NEXT request)

API of HttpServletResponse

```
public void sendRedirect(String redirectLoc)
```

```
eg : resp.sendRedirect("s2");
```

**IMPORTANT :**

**WC -- throws**

```
java.lang.IllegalStateException: Cannot call sendRedirect() after the response has been committed(eg :  
pw.flush(),pw.close()...)
```

**Technique # 2 : Session tracking based on HttpSession API**

**In this technique :**

Entire state of the client is not saved on client side , instead saved on the server side data structure (Http Sesion object) BUT the key to this Http Session object is STILL sent to client in form of a cookie.(cookie management is done by WC)

Servlet programmer can store/restore java objects directly under the session scope(API : setAttribute/getAttribute)

Above mentioned , disadvantages ---0, 1 & 2 are reomved.

BUT entire session tracking again fails , if cookies are disabled.

Steps for javax.servlet.http.HttpSession i/f based session tracking.

**1. Get Http Session object from WC**

API of HttpServletRequest ---

```
HttpSession getSession()
```

Meaning --- Servlet requests WC to either create n return a NEW HttpSession object(for new clnt) or ret the existing one from WC's heap for existing client.

HttpSession --- i/f from javax.servlet.http

In case of new client :

```
HttpSession<String,Object> --empty map
```

```
String,Object ---- (entry)= attribute
```

OR

```
HttpSession getSession(boolean create)
```

**2. : How to save data in HttpSession?(scope=entire session)**

API of HttpSession i/f

```
public void setAttribute(String attrName, Object attrVal)
```

equivalent to map.put(k,v)

```
eg : hs.setAttribute("cart", l1);
```

3. For retrieving session data (getting attributes)

```
public Object getAttribute(String attrName)
```

4. To get session ID (value of the cookie whose name is jsessionid -- unique per client)

```
String getId()
```

#### **4.5 How to remove attribute from the session scope?**

```
public void removeAttribute(String attrName)
```

#### **5. How to invalidate session?**

HttpSession API

```
public void invalidate()
```

(WC marks HS object on the server side for GC --- BUT cookie is NOT deleted from client browser)

#### **6. HttpSession API**

```
public boolean isNew()
```

Returns true for new client & false for existing client.

7. How to find all attr names from the session ?

```
public Enumeration<String> getAttributeNames()
```

-- returns java.util.Enumeration of attr names.

8. Default session timeout value for Tomcat = 30 mins

How to change session timeout ?

HttpSession i/f method

```
public void setMaxInactiveInterval(int secs)
```

```
eg : hs.setMaxInactiveInterval(300); --for 5 mins .
```

OR via xml tags in web.xml

```
<session-config>
```

```
    <session-timeout>5</session-timeout>
```

```
</session-config>
```

**NOTE :**

#### **What is an attribute ?**

attribute = server side object (entry/mapping = key value pair)

who creates server side attrs ? -- web developer (servlet or JSP prog)

Each attribute has --- attr name (String) & attr value (java.lang.Object)

Attributes can exist in one of 3 scopes --- req. scope, session scope or application scope

1. Meaning of req scoped attr = attribute is visible for current req.
2. Meaning of session scoped attr = attribute is visible for current session.(shared across multiple reqs coming from SAME clnt)
3. Meaning of application scoped attr = attribute is visible for current web appln.(shared across multiple reqs from ANY clnt BUT for the SAME web application)

## Regarding SERVLET CONFIG

A servlet specific configuration object used by a servlet container to pass information to a servlet during initialization.

1. Represents Servlet specific configuration.

Defined in javax.servlet.ServletConfig -- interface.

2. Who creates its instance ?

Web container(WC)

3. **When** ?

After WC creates servlet instance(via def constr), ServletConfig instance is created & then it invokes init() method of the servlet.

4. Usage

To store servlet specific init parameters.

(i.e the init-param is accessible to one servlet only or you can say that the init-param data is private for a particular servlet.)

5. Where to add servlet specific init parameters?

Can be added either in web.xml or @WebServlet annotation.

### XML Tags

```
<servlet>
    <servlet-name>init</servlet-name>
    <servlet-class>ex.TestInitParam</servlet-class>
    <init-param>
        <param-name>name</param-name>
        <param-value>value</param-value>
    </init-param>
</servlet>

<servlet-mapping>
    <servlet-name>init</servlet-name>
    <url-pattern>/test_init</url-pattern>
</servlet-mapping>
```

6. How to access servlet specific init params from a servlet ?

## 6.1 Override init() method

## 6.2 Get ServletConfig

Method of Servlet i/f

```
public ServletConfig getServletConfig()
```

## 6.3 Get the init params from ServletConfig

Method of ServletConfig i/f

String getInitparameter(String paramName) : returns the param value.

## Regarding javax.servlet.ServletContext (i/f)

1. Defined in javax.servlet package.

2. Who creates its instance -- WC

3. When -- @ Web application (=context) deployment time

NOTE : The ServletContext object is contained within the ServletConfig object, which the WC provides the servlet when the servlet is initialized.

4. How many instances ? --one per web application

5. Usages

### 5.1 Server side logging

API public void log(String msg)

### 5.2 To create context scoped attributes

API public void setAttribute(String nm, Object val)

NOTE : Access them always in thread safe manner

### 5.3 To access global(scope=entire web application) parameters

## How to add context scoped parameters ?

In web.xml

```
<context-param>
```

```
  <param-name>name</param-name>
```

```
  <param-value>value</param-value>
```

```
</context-param>
```

How to access these params in a Servlet ?

(can be accessed from init method onwards)

## 1. Get ServletContext

API of GenericServlet

ServletContext getServletContext() --method inherited from GenericServlet

## 2. ServletContext API

String getInitparameter(String paramName) : returns the param value.

eg : ctx param name : user\_name value : abc

In the Servlet : `getServletContext().getInitparameter("user_name")` ---abc

#### 5.4 Creating request dispatcher

H.W

---

### What is a Servlet Listener(or web application listener)?

During the lifetime of a typical web application, a number of events take place.

eg : requests are created or destroyed.

sessions are created & destroyed

Contexts(web apps) are created & destroyed.

request or session or context attributes are added, removed, or modified etc.

The Servlet API provides a number of listener interfaces that one can implement in order to react to these events.

eg : Event Listener i/f

1. ServletRequestListener
2. HttpSessionListener
3. ServletContextListener

#### Steps

1. Create a class , implementing from Listener i/f.
2. Register it with WC
- 2.1 @WebListener annotation(class level)

OR

- 2.2 XML tags in web.xml

```
<listener>
```

```
<listener-class>F.Q cls name </listener-class>
```

```
</listener>
```

---

### What is JSP? (Java server pages)

Dynamic Web page (having typically HTML 5 markup) , can embed Java code directly.

Dynamic web component , whose life-cycle is managed by WC(JSP container/Servlet container/Servlet engine)

- **WHY JSP?**

1. JSP allows developer to separate presentation logic(dyn resp generation) from Business logic or data manipulation logic.

Typically JSPs -- used for P.L(presentation logic)

Java Beans or Custom Tags(actions) --- will contain Business logic.

2. Ease of development --- JSP pages are auto. translated by W.C in to servlet & compiled & deployed.

3. Can use web design tools -- for faster development (RAD --rapid application development) tools.

JSP API

jsp-api.jar --- <tomcat>/lib

Contains JSP API implementation classes.

0. javax.servlet.Servlet -- super i/f

1. javax.servlet.jsp.JspPage -- extends Servlet i/f

1.1 public void jspInit()

1.2 public void jspDestroy()

Can be overridden by JSP page author

2. Further extended by javax.servlet.jsp.HttpJspPage

2.1 public void \_jspService(HttpServletRequest rq,HttpServletResponse rs) throws ServletException,IOException.

Never override \_jspService ---JSP container auto translates JSP tags (body) into \_jspService.

### **JSP life-cycle**

1. Clnt sends the 1st request to the JSP (test.jsp)

2. Web-container invokes the life cycle for JSP

3. Translation Phase : handled by the JSP container.

I/p : test.jsp O/p : test\_jsp.java (name : specific to the Tomcat container)

Meaning : .jsp is translated into corresponding servlet page(.java)

Translation time errs : syntactical errs in using JSP syntax.

In case of errs : life-cycle is aborted.

4. Compilation Phase : handled by the JSP container.

I/p : Translated servlet page(.java) O/p : Page Translation class(.class)

Meaning : servlet page auto. compiled into .class file

Compilation time errs: syntacticle errs in generated Java syntax.

5. Request processing phase / Run time phase. : typically handled by the Servlet Container.

6. S.C : will try to locate,load,instantiate the generated servlet class.

7. The 1st it calls : public void jspInit() : one time inits can be performed.(jspInit available from javax.servlet.jsp.JspPage)

8. Then it will call follwing method using thrd created per clnt request :

public void \_jspService(HttpServletRequest Rq,HttpServletResponse) throws ServletException,IOException(API avlble from javax.servlet.jsp.HttpJspPage)

When `_jspService` returns, thread's run method is over & thread returns to the pool, where it can be used for servicing some other or same client's request.

9.. At the end ...(server shutting down or re-deployment of the context) : the S.C calls

```
public void jspDestroy()
```

After this : translated servlet page class instance will be GCed....

10 For 2nd request onwards ..... : SC will invoke step 8 onwards.

JSP 2.0/2.1/2.2 syntax

## 1. JSP comments

### 1.1 server side comment

syntax : `<%-- comment text --%>`

significance : JSP translator & compiler ignores the commented text.

### 1.2 client side comment

syntax : `<!-- comment text -->`

significance : JSP translator & compiler does not ignore the commented text BUT client browser will ignore it.

2. JSP's implicit objects (available only to `_jspService`) -- available to scriptlets, expressions

2.1 `out` - `javax.servlet.jsp.JspWriter` : represents the buffered writer stream connected to the client via `HttpServletResponse` (similar to your `PW` in servlets)

Has the same API as `PW` (except `printf`)

usage eg : `out.print("some text sent to client");`

2.2 `request` : `HttpServletRequest` (same API)

2.3 `response` : `HttpServletResponse`

2.4 `config` : `ServletConfig` (used for passing init params)

2.4 `session` : `HttpSession` (By def. all JSPs participate in session tracking i.e session object is created)

2.5 `exception` : `java.lang.Throwable` (available only to error handling pages)

2.6 `pageContext` : current page environment : `javax.servlet.jsp.PageContext` (this class stores references to page specific objects viz -- `exception`, `out`, `config`, `session`)

2.7 `application` : `ServletContext` (used for Request dispatching, server side logging, for creating context listeners, to avail context params, to add/get context scoped attributes)

2.8 `page` --- current translated page class instance created for 'this' JSP

3. Scripting elements : To include the Java content within JSP : to make it dynamic.

3.1 Scriptlets : can add the Java code directly . AVOID scriptlets . (till JavaBeans & custom tags or JSTL, we will use the scriptlets to add : Req. processing logic, B.L & P.L)

syntax : `<% java code..... %>`

location inside the translated page : within `_jspService`

usage : till Java beans or custom tags are introduced : scriptlets used for control flow/B.L/req. proc. logic

3.2 JSP expressions :

syntax : `<%= expr to evaluate %>`

--Evaluates an expression --converts it to string --send it to client browser.

eg : `<%= new Date() %>`

expr to evaluate : java method invocation which returns a value OR

const expr or attributes(getAttribute) or variables(instance vars or method local)

location inside the translated page : within `_jspService`

**significance : the expr gets evaluated---> to string -> automatically sent to client browser.**

eg `<%= new Date() %>`

eg `<%= request.getAttribute("user_dtls") %>`

`<%= 12*34*456 %>`

`<%= session.getAttribute("user_dtls") %>`

`<%= session.setAttribute("nm",1234) %>` -- compiler error

`<%= session.getId() %>`

Better alternative to JSP Expressions : EL syntax (Expression Lang. : available from JSP 1.2 onwards)

syntax : `${expr to evaluate}` (to be added directly in body tag)

EL syntax will evaluate the expr ---toString --sends it client browser.

JSP implicit object --- request,response,session.....---accessible from scriptlets & JSP exprs. ---

EL implicit objects --- can be accessible only via EL syntax

param = map of request parameters

pageScope=map of page scoped attrs

requestScope=map of request scoped attrs

sessionScope=map of session scoped attrs

applicationScope=map of application(=context) scoped attrs

pageContext --- instance of PageContext's sub class

cookie -- map of cookies(cookie objects)

initParam -- map of context params.

---available ONLY to EL syntax `${...}`

---to be added directly within `<body> ...</body>`

eg : `${param.user_nm}` ---param.get("user\_nm") --value --to string ---> client

request.getParameter("user\_nm") --value --to string ---> client

`${requestScope.abc}` ---request.getAttribute("abc") ---to string --sent to client browser.

eg : suppose ctx scoped attr --- loan\_scheme

`${applicationScope.loan_scheme}` --- getServletContext().getAttribute("loan\_scheme") ---to string --sent to client

`${abc} ---`  
`pageContext.getAttribute("abc") ---not null -- to string -clnt`  
`null`  
`--request.getAttribute("abc") -- not null -- to string -clnt`  
`null`  
`session.getAttribute("abc") ---`  
`null`  
`getServletContext().getAttirbute("abc") --not null -- to string -clnt`  
`null ---BLANK to clnt browser.`  
`eg : ${sessionScope.nm}`

`${pageContext.session.id}`  
`--pageContext.getSession().getId() --- val of JessionId cookie w/o java code.`  
`${pageContext.request.contextPath} ---/day5_web`  
`${pageContext.session.maxInactiveInterval}`  
`----`  
`${param}`  
`{user_nm=asdf, user_pass=123456}`

`eg : ${param.f1} ----> request.getParameter("f1").toString()----> sent to browser`  
`param ----map of req parameters.`  
`param : req. param map`  
`${requestScope.abc} ----- out.print(request.getAttribute("abc").toString())`  
`${abc} -----pageCotext.getAttribute("abc")----null ---request ---session---application ---null ---EL prints blank.`

### 3.3 JSP declarations (private members of the translated servlet class)

syntax : `<%! JSP declaration block %>` (outside `<body>`)

Usage : 1. for creating page scoped java variables & methods (instance vars & methods/static members)

2. Also can be used for overriding life cycle methods (jspInit,jspDestroy)

location inside the translated page : outside of `_jspService` (directly within JSP's translated class)

-----

**JSP Directives** --- commands/messages for JSP Engine(=JSP container=WC) -- to be used @Translation time.

## Syntax ---

<%@ Directive name attrList %>

### 1. page directive

--- all commands applicable to current page only.

#### Syntax

<%@ page import="comma separated list of pkgs" contentType="text/html" %>

eg -- <%@ page import="java.util.\*,java.text.SimpleDateFormat" contentType="text/html" %>

#### Imp page directive attributes

### 1. import --- comma separated list of pkgs

### 2. session --- boolean attribute. def=true.

To disable session tracking, specify session="false"

### 3. errorPage="URI of err handling page" ---

tells WC to forward user to err handler page.

### 4. isErrorPage="true|false" def = false

If u enable this to true--- one can access 'exception' implicit object from this page.

This exception obj is stored under current page ---i.e under pageContext (type=javax.servlet.jsp.PageContext -- class which represents curnt JSP)

EL expression to display error mesg

`${pageContext.exception.message}`

-- evals to `pageContext.getException().getMessage()`

## Additional EL syntax

EL syntax to be used in error handling pages

ERR causing URI : `${pageContext.errorData.requestURI}` <br/>

ERR code : `${pageContext.errorData.statusCode}` <br/>

ERR Mesg : `${pageContext.exception.message}` <br/>

Throwable : `${pageContext.errorData.throwable}` <br/>

Throwable Root cause: `${pageContext.errorData.throwable.cause}`

### 5. isThreadSafe="true|false" default=true. "true" is recommended

true=>informing WC--- JSP is already written in thrd -safe manner ---- DONT apply thrd safety.

false=>informing WC --- apply thrd safety.

(NOT recommended) ---WC typically marks entire service(servlet scenario) or `_jspService` in JSP scenarion --- synchronized. --- this removes concurrent handling of multiple client request --so not recommended.

What is reco? --- `isThreadSafe=true(def.)` --- identify critical code--wrap it in synchronized block.

eg ---Context scoped attrs are inherently thrd -un safe. So access them always from within synched block.

### Equivalent step in Servlet

Servlet class can imple. tag i/f -- javax.servlet.SingleThreadModel(DEPRECATED) -- WC ensures only 1thread (representing clnt request) can invoke service method. --NOT NOT recommended.

2. include directive

```
<%@ include file="URI of the page to be included" %>
```

Via include directive ---- contents are included @ Translation time.--- indicates page scope(continuation of the same page).

Typically used -- for including static content (can be used to include dyn conts)

eg ---one.jsp

```
....<%@ include file="two.jsp" %>
```

two.jsp.....

-----

### JSP actions ---- commands/mesgs meant for WC

to be interpreted @ translation time & applied @ req. processing time.(run time)

Syntax ---standard actions --implementation classes are present in jsp-api.jar.

```
<jsp:actionName attr list>Body of the tag/action
```

```
</jsp:actionName>
```

### JSP Using Java beans

#### Why Java Beans

---1. allows prog to separate B.L in JBs.(Req processing logic, Page navigation & resp generation will be still part of JSP)

JBs can store conversational state of clnt(JB 's properties will reflect clnt state) + supplies Business logic methods.

2. simple sharing of JBS across multiple web pages---gives rise to re-usability.

3. Auto. translation between req. params & JB props(string--->primitive data types auto. done by WC)

#### What is JB?

1. pkged public Java class

2. Must have def constr.(MUST in JSP using JB scenario)

3. Properties of JBs --- private, non-static , non-transient Data members --- equivalent to request params sent by clnt.(Prop names MUST match with req params for easy usage)

In proper words --- Java bean props reflect the conversational state of the clnt.

4. per property -- if RW

naming conventions of JB

supply getter & setter.

Rules for setter (Java Bean Naming convention) : strict

```
public void setPropertyName(Type val)
```

Type -- prop type.

```
eg -- private double regAmount;
```

```
public void setRegAmount(double val)
```

```
{...}
```

Rules for getter

```
public Type getPropertyName()
```

Type -- prop type.

```
eg -- public double getRegAmount(){...}
```

5. Business Logic --- methods

public methods --- no other restrictions

-----

### Using Java Beans from JSP Via standard actions

1. <jsp:useBean id="BeanRef name" class="F.Q. Bean class name" scope="page|request|session|application"/>

def = page scope.

pre-requisite --- JB class exists under <WEB-INF>/classes.

JB = server side obj (attribute), attr name --- bean id, attr val -- bean inst., can be added to any scope using scope attribute.

eg :

eg --- beans.Userbean

props --- email,pass

setters/getters

B.L mehod -- for validation

Usage ---

```
<jsp:useBean id="user" class="beans.UserBean" scope="session"/>
```

W.C invokes JB life-cycle

1. WC chks if specified Bean inst alrdy exists in specified scope

java api --- request.getAttribute("user")

---null=>JB doesn't exist

---loc/load/inst JB class

```
UserBean u1=new UserBean();
```

--add JB inst to the specified scope

java api -- request.setAttribute("user",u1);

--- not-null -- WC continues....

## 2. JSP using JB action

2.1 <jsp:setProperty name="Bean ref Name" property="propName" value="propVal---static/dyn" />

Usage--

```
<jsp:setProperty name="user" property="email"
value="a@b"/>
```

WC invokes --- session.getAttribute("user").setEmail("a@b");

```
<jsp:setProperty name="user" property="email"
value="<%= request.getParameter("f1") %>"/>
```

OR via EL

```
<jsp:setProperty name="user" property="email"
value="${param.f1}"/>
```

WC invokes ---

session.getAttribute("user").setEmail(request.getParameter("f1"));

### 2.2

```
<jsp:setProperty name="Bean ref Name" property="propName" param="rq. param name"/>
```

Usage eg --

```
<jsp:setProperty name="user" property="email" param="f1"/>
```

WC invokes ---

((Userbean)request.getAttribute("user")).setEmail(request.getParameter("f1"));

### 2.3

```
<jsp:setProperty name="Bean ref Name" property="*/>
```

usage

```
<jsp:setProperty name="user" property="*/>
```

eg -- If rq. param names are email & password(i.e matching with JB prop names) then ---matching setters(2) will get called

3. <jsp:getProperty name="Bean ref name" property="propName"/>

Usage --

```
<jsp:getProperty name="user" property="email"/>
```

WC ---

session.getAttribute("user").getEmail()--- toString --- sent to clnt browser.

Better equivalent -- EL syntax

`${sessionScope.user.email}` ---

session.getAttribute("user").getEmail()--- toString --- sent to clnt browser.

`${requestScope.user.validUser.email}`

```
request.getAttribute("user").getValidUser().getEmail()
```

```
${pageContext.exception.message}
```

#### 4.JSP std actions related to Request Dispatcher

RD's forward scenario

```
<jsp:forward page="dispatcher URI" />
```

eg : In one.jsp

```
<jsp:forward page="two.jsp"/>
```

WC invokes ---RD rd=request.getRD("two.jsp");

```
rd.forward(request,response);
```

RD's include scenario

```
<jsp:include page="dispatcher URI" />
```

#### **Why JSTL ? JSP standard tag library**

When JSP std actions are in-sufficient to solve B.L

w/o writing scriptlets --- use additional std actions --- supplied as JSTL actions

JSP standard Tag Library

--- has become std part of J2EE 1.5 onwards.

---support exists in form JAR ---

1. jstl-1.2.jar

For using JSTL steps

1.Copy above JAR into ur run-time classpath(copy jars either in <tomcat\_home>/lib OR <web-inf>/lib

2. Use taglib directive to include JSTL tag library into ur JSP pages.

tag=action

tag library=collection of tags

supplier = JSTL vendor(spec vendor=Sun, JAR vendor=Sun/any J2EE compliant web/app server)

jstl.jar --- consist of Tag implementation classes

Tag libr- TLD -- Tag library descriptor -- desc of tags -- how to use tags

```
<%@ taglib uri="URI of JSTL tag lib" prefix="tag prefix" %>
```

eg --- To import JSTL core lib

```
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
```

3. Invoke JSTL tag/action

3.1 eg

```
<c:set var="abc" value="${param.f1}" />
```

---WC

```
pageContext.setAttribute("abc",request.getParameter("f1"))
```

WC invokes --- session.setAttribute("abc",request.getParameter("f1"));

meaning of <c:set> sets the specified attr to specified scope.

```
<c:set var="details" value="${sessionScope.abc}" />
```

WC

```
pageContext.setAttribute("details",session.getAttribute("abc"));
```

2. <c:remove var="abc" scope="request"/>

WC ---request.removeAttribute("abc") ---removes the attr from req scope.

3.2 JB --- ShopBean -- property --

```
private ArrayList<Category> categories; --g & s
```

```
<c:forEach var="cat" items="${sessionScope.shop.listCategories()}">
```

```
    ${cat}<br/>
```

```
</c:forEach>
```

WC invokes ---

```
for(Category cat : session.getAttribute("shop").listCategories())
```

```
    out.print(cat);
```

eg :

```
<c:forEach var="acct" items="${sessionScope.my_bank.acctSummary}">
```

```
    ${acct.acctID} ${acct.type} ${acct.balance} <br/>
```

```
</c:forEach>
```

[http://localhost:8080/day6\\_web/close\\_acct.jsp?acId=101](http://localhost:8080/day6_web/close_acct.jsp?acId=101)

```
<input type="submit" name="btn" value="Withdraw"
```

```
        formaction="transactions.jsp" /></td>
```

```
<td><input type="submit" name="btn" value="Deposit"
```

```
        formaction="transactions.jsp" /></td>
```

```
<%
```

```
    request.getParameter("btn").equals("Deposit") ---
```

```
%>
```

```
<c:if test="${boolean val}">
```

```
....
```

```
</c:if>
```

```
<c:if test="${param.btn eq 'Deposit'}">
```

```
    in deposit
```

</c:if>

<c:if test="{param.btn eq 'Withdraw'}">

in withdraw

</c:if>

http://localhost:8080/day6\_web/transactions.jsp?acId=102&amount=500&btn=Deposit

<c:redirect url="{sessionScope.my\_bank.closeAccount()}" />

WC --- response.sendRedirect(session.getAttribute("my\_bank").closeAccount());

-----

logout.jsp

Hello user name

log out mesg

clean up dao

invalidate sess.

auto redirect

### 3.3 JSTL action --- for URL rewriting

<c:url var="attr Name" value="URL to be encoded" scope="page|request|session|application"/>

eg : <c:url var="abc" value="next.jsp" />

WC invokes --- pageContext.setAttribute("abc",resp.encodeURL("next.jsp"));

<a href="{abc}">Next</a>

var -- loop var

items -- any JB 's prop --- array based,coll based (List or set) map based.

### Java syntax

for(Category c : categories)

out.write(c.getName())

### How to set session tm out ?

1. programmatically --- using Java API

From HttpSession --- setMaxInactiveInterval(int secs)

2. declaratively -- either using Java annotations OR using XML config files (web.xml)

=====

## Expression Language implicit variables(case sensitive)

1. **pageContext** : PageContext object (javax.servlet.jsp.PageContext) asso. with current page.

2. **pageScope** - a Map that contains page-scoped attribute names and their values.
3. **requestScope** - a Map that contains request-scoped attribute names and their values.
4. **sessionScope** - a Map that contains session-scoped attribute names and their values.
5. **applicationScope** - a Map that contains application-scoped attribute names and their values.
6. **param** - a Map that contains rq. parameter names to a single String parameter value (obtained by calling `ServletRequest.getParameter(String name)`).
7. **paramValues** - a Map that contains rq. param name to a `String[]` of all values for that parameter (similar to calling `ServletRequest.getParameterValues(name)`)
8. **initParam** - a Map that contains context initialization parameter names and their String value (obtained by calling `ServletContext.getInitParameter(String name)`).

eg : `${initParam.db_drvr}`

9. **cookie** : `Map.Entry` of cookies. (entrySet of cookies)

eg : `${cookie.cookieName.value}`

key ---cookie name

value ---`javax.servlet.http.Cookie`

`${cookie.JSESSIONID.value}`

`---cookie.get("JSESSIONID").getValue()`

10. **To retrieve err details from Error handling page.**

ERR causing URI : `${pageContext.errorData.requestURI}`

ERR code : `${pageContext.errorData.statusCode}`

ERR Mesg : `${pageContext.exception.message}`

Throwable : `${pageContext.errorData.throwable}`

Throwable Root cause: `${pageContext.errorData.throwable.cause}`

eg :

`<c:set var="abc" scope="session" value="Hello User...."/>`

`${sessionScope.abc}`

## Session Tracking technique :

HttpSession + URL rewriting

### Why ????

To develop a web app , independent of cookies , for session tracking.

For tracking the clnt (clnt's session) : the only information, WC needs from the clnt browser is JSessionID value. If clnt browser is not sending it using cookie : Servlet/JSP prog can embed the JSessionID info in each outgoing URL .

What is URL Rewriting : Encoding the URL to contain the JSessionID info.

W.C always 1st chks if JsessionID is coming from cookie, if not ---> then it will chk in URL : if it finds JseesionID from the encoded URL : extracts its value & proceeds in the same manner as earlier.

### How to ?

API :

For URLs generated by clicking link/buttons(clnt pull I) use

HttpServletResponse method

```
public String encodeURL(String origURL)
```

**Rets** : origURL;JSESSIONID=12345

For URLs generated by sendRedirect : clnt pull II : use

HttpServletResponse method

```
public String encodeRedirectURL(String redirectURL)
```

**Rets** : redirectURL;JSESSIONID=12345

=====

### What is Hibernate ?

0. Complete solution to the problem of managing persistence in Java.

1. ORM tool.(Object Relational Mapping) used mainly in data access layer or DAO layer.

2. Provides automatic & transperent persistence.

3. JPA(Java Persistence API) implementor

JPA vs Hibernate

JPA ---standard part of J2EE specification --vendor --J2EE (sun)

Implementation classes -- JAR ---hibenrate core JARs(implementor of JPA)

Provides automatic & transparent persistence framework to store & retrieve data from database.

Open Source Java based framework founded by Gavin King in 2001, hosted on hibernate.org

Currently hosted on sourceforge.net

Java Persistence API (JPA) compliant

Current version Hibernate 5.x

Other popular ORM Frameworks

EclipseLink,iBATIS,Kodo etc.

### WHY Hibernate?

It mediates the applications interaction with a relational database, leaving the developer free to concentrate on the business problem at hand.

J2EE developer does not have to use JDBC API & manage data persistence at RDBMS level.

No need to go to Table/Query/Column level.

One has to bootstrap Hibernate framework , create transient(=not yet persistent) POJOs & then rely entirely on Hibernate frmwork to manage persistence

## Details

There is huge mismatch between Object & Relational world.

Formally referred as -- Object-Relational Impedance Mismatch' (sometimes called the 'paradigm mismatch')

### Important Mismatch Points

1. Granularity
2. Sub Types or inheritance n polymorphism
3. Identity
4. Associations
5. Data Navigation

### Cost of Mismatch

1. SQL queries in Java code
2. Iterating through ResultSet & mapping it to POJOs or entities.
3. SQL Exception handling.
4. Transaction management
5. Caching
6. Connection pooling
7. Boiler plate code

### Hibernate Framework --- popular ORM Tool --- JPA (Java persistence API) provider

Hibernate 4.x --- JPA compliant --- Java persistence API --- Its part of J2EE specifications. ---Is fully JPA compliant

BUT it also has additional services / annotations --- specific to Hibernate.

Dev MUST add hibernate JARs ---while deploying appln on web server. Need not add JPA provider JARs , while working on appln server.

Transparent persistence provider.(As POJOs or Entities are not bound to any Persistence API --- its written completely independent of Persistence Provider.)

--Fully supports OOP features --- association,inheritance & polymorphism

--can persist object graphs , consisting of asso. objects

--caches data which is fetched repeatedly (via L1 & L2 cache) -- thus reduces DB traffic(L1 cache - at session level -- built in. L2 cache - pluggable) (More on caching at end of document)

--supports lazy loading -- thus increases DB performance.

(Meaning --- Lazy fetchingThe associated object or collection is fetched lazily, when its first accessed. This results in a new request to the database (unless the associated object is cached). Eager fetchingThe associated object or collection is fetched together with the owning object, using an SQL outer join, and no further database request is required.

--supports Objectified version of SQL -- HQL --works on objects & properties

--Hibernate usually obtains exactly the right lock level automatically . so developer need not worry about applying Read/Write lock.

### Some basics

1. Hibernate uses runtime reflection to determine the persistent properties of a class.
2. The objects to be persisted (called as POJO or Entity) are defined in a mapping document or marked with annotations.

Either these HBM XML docs or annotations serves to describe the persistent fields and associations, as well as any subclasses or proxies of the persistent object.

3. The mapping documents or annotations are compiled at application startup time and provide the framework with necessary information for a persistent class.

### 4. What is Hibernate config.?

An instance of Hib Configuration allows the application to specify properties and mapping documents to be used at the framework start-up.

The Configuration : initialization-time object.

SessionFactory is created from the compiled collection of mapping documents .

The SessionFactory provides the mechanism for managing persistent classes, the Session interface.

6. A web application or Java SE application will create a single Configuration, build a single instance of SessionFactory and then instantiate multiple Sessions in threads servicing client requests.

**SessionFactory** : immutable and does not reflect any changes done later to the Configuration.

7. The Session class provides the interface between the persistent data store and the application.

The Session interface wraps a JDBC connection, which can be user-managed or controlled by Hibernate.

### Hibernate Session

A Hibernate Session is a set of managed entity instances that exist in a particular data store.

Managing an Entity Instances Life Cycle

You manage entity instances (or POJOs) by invoking operations on the entity/POJO using EntityManager/Session instance.

Entity instances are in one of four states (2 imp aspects of it : its asso. with the hibernate session & sync of its state with the underlying DB)

States : new or transient , managed or persistent, detached, removed.

New entity instances have no persistent identity and are not yet associated with a hib. session (transient)

Managed entity instances have a persistent identity and are associated with a hib. session. (persistent : via save() or saveOrUpdate()) Changes to DB will be done when tx is committed.

Detached entity instances have a persistent identity and are not currently associated with a persistence context/Hib session.

Removed entity instances have a persistent identity, are associated with a persistent context and are scheduled for removal from the data store. (removed via session.delete(obj))

## Introduction to Hibernate Caching

While working with Hibernate web applications we will face so many problems in its performance due to database traffic. That too when the database traffic is very heavy. Actually hibernate is well used just because of its high performance only. So some techniques are necessary to maintain its performance.

Caching is the best technique to solve this problem.

The performance of Hibernate web applications is improved using caching by optimizing the database applications.

The cache actually stores the data already loaded from the database, so that the traffic between our application and the database will be reduced when the application wants to access that data again.

At maximum the application will work with the data in the cache only. Whenever some other data is needed, the database will be accessed. Because the time needed to access the database is more when compared with the time needed to access the cache. So obviously the access time and traffic will be reduced between the application and the database.

Here the cache stores only the data related to current running application. In order to do that, the cache must be cleared time to time whenever the applications are changing.

## Difference in get & load

1. Both use common API (i.e load or get(Class c,Serializable id))

Ret type = T

In get --- if id doesn't exist --- returns null

In load --- if id doesn't exist & u are accessing it from within hib session --- throws ObjectNotFoundException

2. In get --- Hibernate uses eager fetching policy ---- meaning will generate select query always & load the state from DB in persistent POJO ref. --- so even if u access the same from within the session(persistent pojo) or outside (detached) the hib session --- NO EXCEPTION(proxy + state)

3. In load --- Hib uses lazy fetching policy ---- meaning it will, by default NOT generate any select query --- so what u have is ONLY PROXY(wrapper ---with no state loaded from DB) --- on such a proxy --- if u access anything outside the hib session(detached) ----

U WILL GET ---LazyInitializationExc

Fix --- 1. Change fetch type --- to eager (NOT AT ALL reco.=> no caching, disabling L1 cache)

2. If u want to access any POJO in detached manner(i.e outside hib session scope) -

fire non-id get method from within session & then hib has to load entire state from DB ---NO LazyInitializationExc

## Session API update Vs merge

Both methods transition detached object to persistent state.

**Update():**- if you are sure that the session does not contain an already persistent instance with the same identifier then use update to save the data in hibernate. If session has such object with same id, then it throws ---  
org.hibernate.NonUniqueObjectException: a different object with the same identifier value was already associated with the session:

**Merge():**-if you want to save your modifications at any time without knowing about the state of a session then use merge() in hibernate.

Lazy fetching (becomes important in relationships or in Load Vs get)

When a client requests an entity(eg - Course POJO) and its associated graph of objects(eg -Student POJO) from the database, it isn't usually necessary to retrieve the whole graph of every (indirectly) associated

object. You wouldn't want to load the whole database into memory at once;

eg: loading a single Category shouldn't trigger the loading of all Items in that category(one-->many)

-----

### What is Session?

Represents a wrapper around pooled out jdbc connection.

-----

Session object is persistence manager for the

hibernate application

Session object is the abstraction of hibernate

engine for the Hibernate application

Session object provides methods to perform

CRUD operations

Example

- save()                - Inserting the record
- get() / load()      - Retrieving the record
- update()            - Updating the record
- delete()            - Deleting the record

### What is SessionFactory?

-----

It is a factory(provider) of session objects.

We use sessionFactory object to create session

object

It is a heavy weight object, therefore it has to

be created only once for an application(typically @ appln start up time) -- typically one per DB per web application.

It's immutable --- Once SF is created, changes made to hibernate.cfg.xml will not be auto reflected in SF.

## What is Configuration Object ?

-----

Configuration object is used to create the  
SessionFactory object.

Object Oriented Representation of Hibernate  
configuration file and

mapping files(or annotations) is nothing but Configuration object.

When we call configure() method on configuration

object ,hibernate configuration file(hibernate.cfg.xml from run time classpath) and mapping  
files (or resources) are loaded in the memory.

-----

## Why connection pooling?

Java applications should use connection pools because :

- Acquiring a new connection is too expensive

- Maintaining many idle connections is expensive

- Creating prepared statements is expensive

Hibernate provides basic or primitive connection pool -- useful only for classroom testing.

Replace it by 3rd party vendor supplied connection pools(eg Apache or C3P0 or hikari in spring boot) for production  
grade applications.

-----

## Natural Key Vs Surrogate Key

If u have User reg system -- then u have a business rule that --- user email must be distinct. So if u want to make this  
as a prim key --then user will have to supply this during registration.

This is called as natural key. Since its value will be user supplied , u cant tell hibernate to generate it for u---i.e cant  
use @GeneratedValue at all.

Where as -- if u say I will reserve user id only for mapping purposes(similar to serial no ), it need not come from user  
at all & can definitely use hib. to auto generate it for u---this is ur surrogate key & can then use @GeneratedValue.

=====\\

## WHY Hibernate?

### There are many advantages of Hibernate Framework over JDBC

1) Opensource , Lightweight : Hibernate framework is opensource & lightweight.

2) Fast performance: The performance of hibernate framework is fast because cache is internally used in hibernate  
framework. There are two types of cache in hibernate framework first level cache and second level cache. First level  
cache is enabled by default.

Third type of cache is --query level cache.(not implicitly enabled)

3) Database Independent query: HQL (Hibernate Query Language) / JPQL (Java persistence query language) is the object-oriented version of SQL. It generates the database independent queries. So you don't need to write database specific queries. Before Hibernate, If database is changed for the project, we need to change the SQL query as well that leads to the maintenance problem.

4) Automatic table creation: Hibernate framework provides the facility to create the tables of the database automatically. So there is no need to create tables in the database manually.

5) Simplifies complex join: To fetch data form multiple tables is easy in hibernate framework.

eg : To display the course names ordered by desc no of participants (many-to-many)

```
select c.name from dac_courses c inner join course_studs cs on c.id = cs.
```

```
c_id inner join dac_students s on cs.s_id = s.stud_id group by c.id order by count(*) desc;
```

```
JPQL -- select c from Course c join fetch c.students group by c.id order by count(*) desc
```

6) Provides query statistics and database status: Hibernate supports Query cache and provide statistics about query and database status.

7. Hibernate translates checked SQLException to un checked org.hibernate.HibernateException(super cls of all hibernate related errs)

---so that prog doesn't have to handle excs.

-----

### **Advantages of hibernates:**

1. Hibernate supports Inheritance, Associations, Collections.

2. In hibernate if we save the derived class object, then its base class object will also be stored into the database, it means hibernate supporting inheritance

3. Hibernate supports relationships like One-To-Many,One-To-One, Many-To-Many-to-Many, Many-To-One

4. This will also supports collections like List,Set,Map (Only new collections)

5. In jdbc all exceptions are checked exceptions, so we must write code in try, catch and throws, but in hibernate we only have Un-checked exceptions, so no need to write try, catch, or no need to write throws. Actually in hibernate we have the translator which converts checked to Un-checked ;)

6. Hibernate has capability to generate primary keys automatically while we are storing the records into database

7. Hibernate has its own query language, i.e hibernate query language which is database independent

So if we change the database, then also our application will works as HQL is database independent

HQL contains database independent commands

8. While we are inserting any record, if we dont have any particular table in the database, JDBC will rises an error like View not exist, and throws exception, but in case of hibernate, if it not found any table in the database this will create the table for us ;)

9. Hibernate supports caching mechanism by this, the number of round trips between an application and the database will be reduced, by using this caching technique an application performance will be increased automatically.

Hibernate supports annotations, apart from XML

10. Hibernate provided Dialect classes, so we no need to write sql queries in hibernate, instead we use the methods provided by that API.

11. Getting pagination in hibernate is quite simple.

## Hibernate API

### 0. SessionFactory API

#### getCurrentSession vs openSession

public Session openSession() throws HibernateExc

opens new session from SF, which has to be explicitly closed by prog.

public Session getCurrentSession() throws HibernateExc

Opens new session, if one doesn't exist, otherwise continues with the existing one.

Gets automatically closed upon Tx boundary or thread over (since current session is bound to current thread -- mentioned in hibernate.cfg.xml property ---current\_session\_context\_class ---thread)

### 1. CRUD logic (save method)

API (method) of org.hibernate.Session

public Serializable save(Object o) throws HibernateException

I/P ---transient POJO ref.

save() method auto persists transient POJO on the DB (upon committing tx) & returns unique serializable ID generated by (currently) hib framework.

### 2. Hibernate session API -- for data retrieval

API (method) of org.hibernate.Session

public <T> T get(Class<T> c, Serializable id) throws HibernateException

T -- type of POJO

Returns --- null -- if id is not found.

returns PERSISTENT pojo ref if id is found.

Usage of Hibernate Session API's get()

int id=101;

BookPOJO b1=hibSession.get(BookPOJO.class,id);

BookPOJO b1=(BookPOJO)hibSession.get(Class.forName("pojos.BookPOJO"),id);

### 2. Display all books info :

using HQL -- Hibernate Query Language/JPQL --- Objectified version of SQL --- where table names will be replaced by POJO class names & table col names will be replaced by POJO property names.

(JPA--- Java Persistence API compliant syntax --- JPQL )

eg --- HQL --- "from BookPOJO"

eg JPQL -- "select b from BookPOJO b"

## 2.1 Create Query Object --- from Session i/f

<T> org.hibernate.query.Query<T> createQuery(String queryString, Class<T> resultType)

eg : Query<Book> q=hs.createQuery(hql/jpql,Book.class);

## 2.2. Execute query to get List of selected PERSISTENT POJOs

API of org.hibernate.query.Query i/f

(Taken from javax.persistence.TypedQuery<T>)

List<T> getResultList()

Execute a SELECT query and return the query results as a generic List<T>.

T -- type of POJO / Result

eg : hs,tx

String jpql="select b from Book b";

try {

List<Book> l1=hs.createQuery(jpql,Book.class).getResultList();

}

Usage ---

String hql="select b from BookPOJO b";

List<BookPOJO> l1=hibSession.createQuery(hql).getResultList();

## 3. Passing IN params to query. & execute it.

Objective : Display all books from specified author , with price < specified price.

API from org.hibernate.query.Query i/f

Query<R> setParameter(String name,Object value)

Bind a named query parameter using its inferred Type.

name -- query param name

value -- param value.

String hql="select b from BookPOJO b where b.price < :sp\_price and b.author = :sp\_auth";

### How to set IN params ?

org.hibernate.query.Query<T> API

public Query<T> setParameter(String pName,Object val)

List<Book> l1 =

hibSession.createQuery(hql,Book.class).setParameter("sp\_price",user\_price).setParameter("sp\_auth",user\_auth).getResultList();

Objective --Offer discount on all old books

i/p -- date , disc amt

**4.Updating POJOs** --- Can be done either with select followed by update or ONLY with update queries(following is eg of 2nd option--Bulk update scenario)

Objective : dec. price of all books with author=specified author.

```
String jpql = "update BookPOJO b set b.price = b.price - :disc where b.author = :au and b.publishDate < :dt ";
```

set named In params

exec it (executeUpdate) ---

```
int updateCount= hs.createQuery(hql).setParameter("disc", disc).setParameter("dt", d1).executeUpdate();
```

---This approach is typically NOT recommended often, since it bypasses L1 cache . Cascading is not supported. Doesn't support optimistic locking directly.

## 5. Delete operations.

API of org.hibernate.Session

--void delete(Object object) throws HibernateException

---POJO is marked for removal , corresponding row from DB will be deleted after comitting tx & closing of session.

OR

5.5

**One can use directly "delete HQL" & perform deletions.(Bulk delete)**

eg

```
int deletedRows = hibSession.createQuery ("delete Subscription s WHERE s.subscriptionDate < :today").setParameter ("today", new Date ()).executeUpdate ();
```

## API of org.hibernate.query.Query<T>

1. Iterator iterate() throws HibernateException

Return the query results as an Iterator. If the query contains multiple results per row, the results are returned Object[].

Entities returned --- in lazy manner

Pagination

2. Query setMaxResults(int maxResults)

Set the maximum number of rows to retrieve. If not set, there is no limit to the number of rows retrieved.

3. Query setFirstResult(int firstResult)

Set the first row to retrieve. If not set, rows will be retrieved beginnning from row 0. (NOTE row num starts from 0)

```
eg --- List<CustomerPOJO> l1=sess.createQuery("select c from CustomerPOJO c").setFirstResult(30).setMaxResults(10).list();
```

#### 4. How to count rows & use it in pagination techniques?

```
int pageSize = 10;

String countQ = "Select count (f.id) from Foo f";

Query countQuery = session.createQuery(countQ);

Long countResults = (Long) countQuery.uniqueResult();

int lastPageNumber = (int) ((countResults / pageSize) + 1);


Query selectQuery = session.createQuery("From Foo");
selectQuery.setFirstResult((lastPageNumber - 1) * pageSize);
selectQuery.setMaxResults(pageSize);
List<Foo> lastPage = selectQuery.list();
```

#### 5. org.hibernate.query.Query API

<T> T getSingleResult()

Executes a SELECT query that returns a single typed result.

Returns: Returns a single instance(persistent) that matches the query.

Throws:

NoResultException - if there is no result

NonUniqueResultException - if more than one result

IllegalStateException - if called for a Java Persistence query language UPDATE or DELETE statement

#### 6. How to get Scrollable Result from Query?

ScrollableResults scroll(ScrollMode scrollMode) throws HibernateException

Return the query results as ScrollableResults. The scrollability of the returned results depends upon JDBC driver support for scrollable ResultSets.

Then can use methods of ScrollableResults ---first,next,last,scroll(n) .

#### 7. How to create Named query from Session i/f?

**What is a named query ?**

Its a technique to group the HQL statements in single location(typically in POJOS) and lately refer them by some name whenever need to use them. It helps largely in code cleanup because these HQL statements are no longer scattered in whole code.

Fail fast: Their syntax is checked when the session factory is created, making the application fail fast in case of an error.

Reusable: They can be accessed and used from several places which increase re-usability.

eg : In POJO class, at class level , one can declare Named Queries

@Entity

@NamedQueries

```
{{@NamedQuery(name=DepartmentEntity.GET_DEPARTMENT_BY_ID, query="select d from DepartmentEntity d where d.id = :id")}}
```

```
public class Department{....}
```

Usgae

```
Department d1 = (Department)
```

```
session.getNamedQuery(DepartmentEntity.GET_DEPARTMENT_BY_ID).setInteger("id", 1);
```

8. How to invoke native sql from hibernate?

```
Query q=hs.createSQLQuery("select * from books").addEntity(BookPOJO.class);
```

```
l1 = q.list();
```

9. Hibernate Criteria API

A powerful and elegant alternative to HQL

Well adapted for dynamic search functionalities where complex Hibernate queries have to be generated 'on-the-fly'.

Typical steps are -- Create a criteria for POJO, add restrictions , projections ,add order & then fire query(via list() or uniqueResult())

10. For composite primary key

Rules on prim key class

Annotation -- @Embeddable (& NOT @Entity)

Must be Serializable.

Must implement hashCode & equals as per general contract.

In Owning Entity class

Add usual annotation -- @Id.

### 1.1 Testing core api

**persist** ---

```
public void persist(Object transientRef)
```

---persists transient POJO .

if u give some non-null id (existing or non-existing) while calling persist(ref) --gives exc

org.hibernate.PersistentObjectException: detached entity passed to persist:

why its taken as detached ? ---non null id.

2.

public Serializable save(Object ref)

save --- if u give some non-null id(existing or non-existing) while calling save(ref) --doesn't give any exc.

Ignores ur passed id & creates its own id & inserts a row.

### 3. saveOrUpdate

public void saveOrUpdate(Object ref)

--either inserts/updates or throws exc.

null id -- fires insert (works as save)

non-null BUT existing id -- fires update (works as update)

non-null BUT non existing id -- throws StateStateException --to indicate that we are trying to delete or update a row that does not exist.

3.5

### merge

public Object merge(Object ref)

I/P -- either transient or detached POJO ref.

O/P --Rets PERSISTENT POJO ref.

null id -- fires insert (works as save)

non-null BUT existing id -- fires update (select , update)

non-null BUT non existing id -- no exc thrown --Ignores ur passed id & creates its own id & inserts a row.(select,insert)

### 4. get vs load

& LazyInitilalizationException

### 5. update

Session API

public void update(Object object)

Update the persistent instance with the identifier of the given detached instance.

I/P --detached POJO containing updated state.

Same POJO becomes persistent.

### Exception associated :

1. org.hibernate.TransientObjectException: The given object has a null identifier:

i.e while calling update if u give null id. (transient ----X ---persistent via update)

2. org.hibernate.StateStateException --to indicate that we are trying to delete or update a row that does not exist.

3.

org.hibernate.NonUniqueObjectException: a different object with the same identifier value was already associated with the session

6. public Object merge(Object ref)

Can Transition from transient --> persistent & detached ---> persistent.

Regarding Hibernate merge

1. The state of a transient or detached instance may also be made persistent as a new persistent instance by calling merge().

## 2. API of Session

Object merge(Object object)

3.

Copies the state of the given object(can be passed as transient or detached) onto the persistent object with the same identifier.

3.If there is no persistent instance currently associated with the session, it will be loaded.

4.Return the persistent instance. If the given instance is unsaved, save a copy of and return it as a newly persistent instance. The given instance does not become associated with the session.

5. will not throw NonUniqueObjectException --Even If there is already persistence instance with same id in session.

7.public void evict(Object persistentPojoRef)

It detaches a particular persistent object

detaches or disassociates from the session level cache(L1 cache)

(Remove this instance from the session cache. Changes to the instance will not be synchronized with the database. )

8.

## void clear()

When clear() is called on session object all the objects associated with the session object(L1 cache) become detached.

But Database Connection is not returned to connection pool.

(Completely clears the session. Evicts all loaded instances and cancel all pending saves, updates and deletions)

## 9. void close()

When close() is called on session object all

the persistent objects associated with the session object become detached(l1 cache is cleared) and also closes the Database Connection.

## 10. void flush()

When the object is in persistent state , whatever changes we made to the object

state will be reflected in the database only at the end of transaction.

BUT If we want to reflect the changes before the end of transaction

(i.e before committing the transaction )

call the flush method.

(Flushing is the process of synchronizing the underlying DB state with persistable state of session cache )

### 11. boolean contains(Object ref)

The method indicates whether the object is associated with session or not.(i.e is it a part of l1 cache ?)

12.

void refresh(Object ref) -- ref --persistent or detached

This method is used to get the latest data from database and make corresponding modifications to the persistent object state.

(Re-reads the state of the given instance from the underlying database

=====

## Hibernate Association

### The two rules, for bidirectional one-to-one associations

1 The @JoinColumn annotation goes on the mapping of the entity that is mapped to the table containing the join column, or the owner of the relationship. This might be on either side of the association.

2 The mappedBy element should be specified in the @OneToOne annotation in the entity that does not define a join column, or the inverse side of the relationship.

Illegal to have a bidirectional association that had mappedBy on both sides.

Incorrect not have it on either side.

Hibernate will assume each side was the owner and each will have a join column.

-----

When an entity is associated with a Collection of other entities, it is in form of a one-to-many mapping.

When a relationship is bidirectional, there are actually two mappings, one for each direction.

A bidirectional one-to-many relationship always implies a many-to-one mapping back to the source.

eg : Course & Students

In this , there is a one-to-many mapping from Course -----> Student

and a many-to-one mapping from Student -----> Course.

### Which table must have a foreign key ?

When a Course entity has an number of Student entities stored in its collection, there is no definite way to store those references in the database table.

Instead Student table MUST have foreign keys back to the Course

So the one-to-many association is almost always bidirectional and never the owning side.

In Course entity u need to map the Students with @OneToMany annotation.

This doesn't have foreign key , so its an inverse side of relationship.

Since this is the inverse side of the relationship, MUST include the mappedBy attribute.

eg : In Course Entity

**mappedBy** -- must refer to the prop name in the associated table --to specify ownership of the asso.

@OneToMany(cascade = CascadeType.ALL,mappedBy = "course")

students getter. (getStudents)

---mappedBy tells hibernate that instead of having a separate join table --map students by course i.e course = name of the property appearing in Student class , anno by @ManyToOne

In Student Entity

@ManyToOne

@JoinColumn(name = "course\_id")

getter for course

### Rules

1. The many-to-one side is the owning side, so the join column is defined on that side.
2. The one-to-many mapping is the inverse side, so the mappedBy element must be used.

-----

### Collection of 3 types --- entities , embeddables & basic types.

In Course 1<---->n Student

the Course entity has a collection of Student instances.

It is called a multivalued relationship.

BUT collections of embeddable and basic types are not relationships; they are simply collections of elements .

They are called element collections.

Relationships define associations between independent entities , whereas element

collections contain objects that are dependent upon the referencing entity, and can be retrieved only through the entity that contains them.

**Annotation Used -- @ElementCollection --mandatory**

BUT then hibernate creates its own table to store these embeddables

If u want to name the table , optional anno is @CollectionTable

@ElementCollection

@CollectionTable(name = "table\_name", joinColumns = @JoinColumn(name = "join\_col\_name"))

Followed by getter of embeddables or basic types.

---

## Entity Vs Value Type

### Entity Types :

1. If an object has its own database identity (primary key value) then it's type is Entity Type.
2. An entity has its own lifecycle. It may exist independently of any other entity.
3. An object reference to an entity instance is persisted as a reference in the database (a foreign key value).

eg : College is an Entity Type. It has it's own database identity (It has primary key).

### Value Types :

1. If an object don't have its own database identity (no primary key value) then it's type is Value Type.
2. Value Type object belongs to an Entity Type Object.
3. It's embedded in the owning entity and it represents the table column in the database.
4. The lifespan of a value type instance is bounded by the lifespan of the owning entity instance.

### Different types of Value Types

Basic, Composite, Collection Value Types :

#### 1. Basic Value Types :

Basic value types are : they map a single database value (column) to a single, non-aggregated Java type.

Hibernate provides a number of built-in basic types.

String, Character, Boolean, Integer, Long, Byte, ... etc

#### 2. Composite Value Types :

In JPA composite types also called Embedded Types. Hibernate traditionally called them Components.

2.1 Composite Value type looks like exactly an Entity, but does not own lifecycle and identifier.

### Annotations Used

#### 1. @Embeddable :

Defines a class whose instances are stored as an intrinsic part of an owning entity and share the identity of the entity. Each of the persistent properties or fields of the embedded object is mapped to the database table for the entity. It doesn't have own identifier.

eg : Address is eg of Embeddable

Student HAS-A Address(eg of Composition --i.e Address can't exist w/o its owning Entity i.e Student)

College HAS-A Address (eg of Composition --i.e Address can't exist w/o its owning Entity i.e College)

BUT Student will have its own copy of Address & so will College(i.e Value Types don't support shared reference)

#### 2. @Embedded :

Specifies a persistent field or property of an entity whose value is an instance of an embeddable class. The embeddable class must be annotated as Embeddable.

eg : Address is embedded in College and User Objects.

#### 3. @AttributesOverride :

Used to override the mapping of a Basic (whether explicit or default) property or field or Id property or field.

In Database tables observe the column names. Student table having STREET\_ADDRESS column and College table having STREET column. These two columns should map with same Address field streetAddress. @AttributeOverride gives solution for this.

To override multiple column names for the same field use @AttributeOverrides annotation.

eg : In Student class :

@Embedded

@AttributeOverride(name="streetAddress", column=@Column(name="STREET\_ADDRESS"))

private Address address;

where , name --POJO property name in Address class

### 3.Collection Value Types :

Hibernate allows to persist collections.

But Collection value Types can be either collection of Basic value types, Composite types and custom types.

eg :

Collection mapping means mapping group of values to the single field or property. But we can't store list of values in single table column in database. It has to be done in a separate table.

eg : Collection of embeddables

#### @ElementCollection

@CollectionTable(name="CONTACT\_ADDRESS", joinColumns=@JoinColumn(name="USER\_ID"))

@AttributeOverride(name="streetAddress", column=@Column(name="STREET\_ADDRESS"))

private List<ContactAddress> address;

eg : collection of basic type

@ElementCollection

@CollectionTable(name="Contacts", joinColumns=@JoinColumn(name="ID"))

@Column(name="CONTACT\_NO")

private Collection<String> contacts;

=====

- **Hibernate API**

#### 0. SessionFactory API

getCurrentSession vs openSession

public Session openSession() throws HibernateExc

opens new session from SF, which has to be explicitly closed by prog.

public Session getCurrentSession() throws HibernateExc

Opens new session , if one doesn't exist , otherwise continues with the existing one.

Gets automatically closed upon Tx boundary or thread over(since current session is bound to current thread -- mentioned in hibernate.cfg.xml property ---current\_session\_context\_class ---thread)

### 1. CRUD logic (save method)

API (method) of org.hibernate.Session

public Serializable save(Object o) throws HibernateException

I/P ---transient POJO ref.

save() method auto persists transient POJO on the DB(upon committing tx) & returns unique serializable ID generated by (currently) hib frmwork.

### 2. Hibernate session API -- for data retrieval

API (method) of org.hibernate.Session

public <T> T get(Class<T> c,Serializable id) throws HibernateException

T -- type of POJO

Returns --- null -- if id is not found.

returns PERSISTENT pojo ref if id is found.

Usage of Hibernate Session API's get()

```
int id=101;
```

```
BookPOJO b1=hibSession.get(BookPOJO.class,id);
```

```
BookPOJO b1=(BookPOJO)hibSession.get(Class.forName("pojos.BookPOJO"),id);
```

### 2. Display all books info :

using HQL -- Hibernate Query Language/JPQL --- Objectified version of SQL --- where table names will be replaced by POJO class names & table col names will be replaced by POJO property names.

(JPA--- Java Persistence API compliant syntax --- JPQL )

eg --- HQL --- "from BookPOJO"

eg JPQL -- "select b from BookPOJO b"

#### 2.1 Create Query Object --- from Session i/f

```
<T> org.hibernate.query.Query<T> createQuery(String queryString,Class<T> resultType)
```

```
eg : Query<Book> q=hs.createQuery(hql/jpql,Book.class);
```

#### 2.2. Execute query to get List of selected PERSISTENT POJOs

API of org.hibernate.query.Query i/f

(Taken from javax.persistence.TypedQuery<T>)

```
List<T> getResultList()
```

Execute a SELECT query and return the query results as a generic List<T>.

T -- type of POJO / Result

```
eg : hs,tx
```

```
String jpql="select b from Book b";

try {

    List<Book> l1=hs.createQuery(jpql,Book.class).getResultList();

}
```

#### Usage ---

```
String hql="select b from BookPOJO b";

List<BookPOJO> l1=hibSession.createQuery(hql).getResultList();
```

### 3. Passing IN params to query. & execute it.

Objective : Display all books from specified author , with price < specified price.

API from org.hibernate.query.Query i/f

```
Query<R> setParameter(String name,Object value)
```

Bind a named query parameter using its inferred Type.

name -- query param name

value -- param value.

```
String hql="select b from BookPOJO b where b.price < :sp_price and b.author = :sp_auth";
```

#### How to set IN params ?

org.hibernate.query.Query<T> API

```
public Query<T> setParameter(String pName,Object val)
```

```
List<Book> l1 =
hibSession.createQuery(hql,Book.class).setParameter("sp_price",user_price).setParameter("sp_auth",user_auth).get
ResultList();
```

Objective --Offer discount on all old books

i/p -- date , disc amt

**4.Updating POJOs** --- Can be done either with select followed by update or ONLY with update queries(following is eg of 2nd option--Bulk update scenario)

Objective : dec. price of all books with author=specified author.

```
String jpql = "update BookPOJO b set b.price = b.price - :disc where b.author = :au and b.publishDate < :dt ";
```

set named In params

exec it (executeUpdate) ---

```
int updateCount= hs.createQuery(hql).setParameter("disc", disc).setParameter("dt", d1).executeUpdate();
```

---This approach is typically NOT recommended often, since it bypasses L1 cache . Cascading is not supported. Doesn't support optimistic locking directly.

### 5. Delete operations.

API of org.hibernate.Session

--void delete(Object object) throws HibernateException

---POJO is marked for removal , corresponding row from DB will be deleted after committing tx & closing of session.

OR

5.5

One can use directly "delete HQL" & perform deletions.(Bulk delete)

eg

```
int deletedRows = hibSession.createQuery ("delete Subscription s WHERE s.subscriptionDate < :today").setParameter ("today", new Date ()).executeUpdate ();
```

API of org.hibernate.query.Query<T>

1. Iterator iterate() throws HibernateException

Return the query results as an Iterator. If the query contains multiple results per row, the results are returned Object[].

Entities returned --- in lazy manner

### Pagination

2. Query setMaxResults(int maxResults)

Set the maximum number of rows to retrieve. If not set, there is no limit to the number of rows retrieved.

3. Query setFirstResult(int firstResult)

Set the first row to retrieve. If not set, rows will be retrieved beginning from row 0. (NOTE row num starts from 0)

eg --- List<CustomerPOJO> l1=sess.createQuery("select c from CustomerPOJO c").setFirstResult(30).setMaxResults(10).list();

### 4.How to count rows & use it in pagination techniques?

```
int pageSize = 10;
```

```
String countQ = "Select count (f.id) from Foo f";
```

```
Query countQuery = session.createQuery(countQ);
```

```
Long countResults = (Long) countQuery.uniqueResult();
```

```
int lastPageNumber = (int) ((countResults / pageSize) + 1);
```

```
Query selectQuery = session.createQuery("From Foo");
```

```
selectQuery.setFirstResult((lastPageNumber - 1) * pageSize);
```

```
selectQuery.setMaxResults(pageSize);
```

```
List<Foo> lastPage = selectQuery.list();
```

5. org.hibernate.query.Query API

<T> T getSingleResult()

Executes a SELECT query that returns a single typed result.

Returns: Returns a single instance(persistent) that matches the query.

**Throws:**

NoResultException - if there is no result

NonUniqueResultException - if more than one result

IllegalStateException - if called for a Java Persistence query language UPDATE or DELETE statement

## 6. How to get Scrollable Result from Query?

ScrollableResults scroll(ScrollMode scrollMode) throws HibernateException

Return the query results as ScrollableResults. The scrollability of the returned results depends upon JDBC driver support for scrollable ResultSets.

Then can use methods of ScrollableResults ---first,next,last,scroll(n) .

## 7. How to create Named query from Session i/f?

### What is a named query ?

Its a technique to group the HQL statements in single location(typically in POJOS) and later refer them by some name whenever need to use them. It helps largely in code cleanup because these HQL statements are no longer scattered in whole code.

Fail fast: Their syntax is checked when the session factory is created, making the application fail fast in case of an error.

Reusable: They can be accessed and used from several places which increase re-usability.

eg : In POJO class, at class level , one can declare Named Queries

@Entity

@NamedQueries

```
{{@NamedQuery(name=DepartmentEntity.GET_DEPARTMENT_BY_ID, query="select d from DepartmentEntity d  
where d.id = :id")}}
```

```
public class Department{....}
```

### Usgae

```
Department d1 = (Department)
```

```
session.getNamedQuery(DepartmentEntity.GET_DEPARTMENT_BY_ID).setInteger("id", 1);
```

## 8. How to invoke native sql from hibernate?

```
Query q=hs.createSQLQuery("select * from books").addEntity(BookPOJO.class);
```

```
l1 = q.list();
```

## 9. Hibernate Criteria API

A powerful and elegant alternative to HQL

Well adapted for dynamic search functionalities where complex Hibernate queries have to be generated 'on-the-fly'.

Typical steps are -- Create a criteria for POJO, add restrictions , projections ,add order & then fire query(via list() or uniqueResult())

## 10. For composite primary key

Rules on prim key class

Annotation -- @Embeddable (& NOT @Entity)

Must be Serializable.

Must implement hashCode & equals as per general contract

In Owning Entity class

Add usual annotation -- @Id.

### 1.1 Testing core api

persist ---

```
public void persist(Object transientRef)
```

---persists transient POJO .

if u give some non-null id (existing or non-existing) while calling persist(ref) --gives exc

org.hibernate.PersistentObjectException: detached entity passed to persist:

why its taken as detached ? ---non null id.

### 2. public Serializable save(Object ref)

save --- if u give some non-null id(existing or non-existing) while calling save(ref) --doesn't give any exc.

Ignores ur passed id & creates its own id & inserts a row.

### 3. saveOrUpdate

```
public void saveOrUpdate(Object ref)
```

--either inserts/updates or throws exc.

null id -- fires insert (works as save)

non-null BUT existing id -- fires update (works as update)

non-null BUT non existing id -- throws StaleStateException --to indicate that we are trying to delete or update a row that does not exist.

### 3.5 merge

```
public Object merge(Object ref)
```

I/P -- either transient or detached POJO ref.

O/P --Returns PERSISTENT POJO ref.

null id -- fires insert (works as save)

non-null BUT existing id -- fires update (select , update)

non-null BUT non existing id -- no exc thrown --Ignores ur passed id & creates its own id & inserts a row.(select,insert)

### 4. get vs load

& LazyInitializationException.

### 5. update

Session API

`public void update(Object object)`

Update the persistent instance with the identifier of the given detached instance.

I/P --detached POJO containing updated state.

Same POJO becomes persistent.

#### **Exception associated :**

1. `org.hibernate.TransientObjectException`: The given object has a null identifier:

i.e while calling update if u give null id. (transient ----X ---persistent via update)

2. `org.hibernate.StaleStateException` --to indicate that we are trying to delete or update a row that does not exist.

3.

`org.hibernate.NonUniqueObjectException`: a different object with the same identifier value was already associated with the session

6. `public Object merge(Object ref)`

Can Transition from transient --> persistent & detached ---> persistent.

Regarding Hibernate merge

1. The state of a transient or detached instance may also be made persistent as a new persistent instance by calling `merge()`.

2. API of Session

`Object merge(Object object)`

3.

Copies the state of the given object(can be passed as transient or detached) onto the persistent object with the same identifier.

3.If there is no persistent instance currently associated with the session, it will be loaded.

4.Return the persistent instance. If the given instance is unsaved, save a copy of and return it as a newly persistent instance. The given instance does not become associated with the session.

5. will not throw `NonUniqueObjectException` --Even If there is already persistence instance with same id in session.

7.`public void evict(Object persistentPojoRef)`

It detaches a particular persistent object

detaches or disassociates from the session level cache(L1 cache)

(Remove this instance from the session cache. Changes to the instance will not be synchronized with the database. )

8. `void clear()`

When `clear()` is called on session object all the objects associated with the session object(L1 cache) become detached.

But Database Connection is not returned to connection pool.

(Completely clears the session. Evicts all loaded instances and cancel all pending saves, updates and deletions)

9. `void close()`

When close() is called on session object all

the persistent objects associated with the session object become detached(l1 cache is cleared) and also closes the Database Connection.

10. void flush()

When the object is in persistent state , whatever changes we made to the object state will be reflected in the database only at the end of transaction.

BUT If we want to reflect the changes before the end of transaction (i.e before committing the transaction )

call the flush method.

(Flushing is the process of synchronizing the underlying DB state with persistable state of session cache )

11. boolean contains(Object ref)

The method indicates whether the object is associated with session or not.(i.e is it a part of l1 cache ?)

12.void refresh(Object ref) -- ref --persistent or detached

This method is used to get the latest data from database and make corresponding modifications to the persistent object state.

(Re-reads the state of the given instance from the underlying database

=====

●

## Spring

### Why Spring

It simplifies Java development. It's one-stop-shop.

Excellent for integration with existing frameworks.

Reduces boiler-plate code.

Allows to build applications using loose coupling achieved via IoC(Inversion of control) & AOP(aspect oriented programming)

### What is it ?

It is a container + framework.

### Why Container

It manages the life cycle of spring beans (eg : controller,rest controller , dao,service)

### Why Framework ?

It provides readymade implementation of patterns & helps in building enterprise applications.

It is the most popular application development framework for enterprise Java. It is used to create high performant, easily testable, and reusable code.

**Spring framework is an open source Java platform.**

Founder is Rod Johnson and was first released under the Apache license in June 2003.

Currently hosted on Pivotal/VMware

## Why Spring

1. Spring is lightweight .The basic version of Spring framework is around 2MB.
2. It supports in developing any Java application, but there are extensions(web MVC) for building web applications on top of the Java EE platform.
3. It helps programmers to make J2EE development easier to use and promotes good programming practices by enabling a POJO-based programming model.
4. Excellent n easy testing support.(Thanks to D.I)
5. Supports smooth integration with ORM
6. Easy integration with web MVC applications including web sockets

(for async communication between server & client)

Spring's web framework is a web MVC framework, which is a great alternative to web frameworks such as Struts

7. It is organized in a modular fashion. Even though it's extensive , you have to worry only about the those modules that you need and ignore the rest.
8. Spring does not re-invent the wheel, instead it makes use of already existing frameworks like Hibernate , making its integration easier.
9. It translates technology-specific exceptions (thrown by JDBC, Hibernate, or JDO, for example) into consistent, unchecked exceptions.
10. It provides a consistent transaction management interface that can support a local transaction (using a single database) as well as global transactions (using JTA over multiple databases).

**Main winning feature of Spring is : loose coupling between the modules.**

## How does it achieve loose coupling ?

1. IoC -- IoC is achieved using Dependency Injection(D.I)
2. Aspect Oriented Programming(AOP)

=====

## Spring Framework Modules

### 1. Core Container

It consists of the Core, Beans, Context, and Expression Language modules

- 1.1 The Core module provides the fundamental parts of the framework, including the IoC based upon Dependency Injection
- 1.2 The Bean module provides BeanFactory , which is a readymade implementation of the factory pattern.
- 1.3 The Context module is based upon the Core and Beans modules .It provides a way to access any spring beans(objects managed by Spring container) which are defined and configured. The ApplicationContext interface is the heart of Context module.
- 1.4 The SpEL module provides a powerful expression language for querying and manipulating an object graph at runtime.

## 2. Data Access/Integration module

The Data Access/Integration layer consists of the JDBC, ORM, OXM, JMS and Transaction modules

2.1 The JDBC module provides a JDBC-abstraction layer that removes the need for boilerplate JDBC related coding.

2.2 The ORM module provides integration layers for popular object-relational mapping APIs, including JPA, JDO, Hibernate, and iBatis.

2.3 The OXM module provides an abstraction layer that supports Object/XML mapping implementations for JAXB, XMLBeans, XStream etc

2.4 JMS module contains features for producing and consuming messages sent across application components

2.5 The Transaction module supports programmatic and declarative transaction management for typically service layer or DAO layer spring beans.

## 3. Web

The Web layer consists of the Web, Web-MVC, Web-Socket, and Web-Portlet modules .

The Web module provides basic web integration features such as MVC support, multipart file-upload functionality, init of IoC container using servlet listeners, web application context, i18N etc.

3.1 The Web-MVC module contains Spring's Model-View-Controller (MVC) implementation for web applications.

3.2 The Web-Socket module provides support for WebSocket-based, two-way communication between the client and the server in web applications.

3.3 The Web-Portlet module provides the MVC implementation to be used in a portlet environment

## Other important Modules

1. The AOP module provides an aspect-oriented programming implementation allowing you to define method-interceptors and pointcuts to cleanly decouple code that implements functionality that should be separated.

2. The Instrumentation module

3. The Messaging module provides support for STOMP as the WebSocket sub-protocol to use in web socket applications.

4. The Test module supports the testing of Spring components with JUnit or TestNG frameworks.

=====

## Spring 4 DB Transactions

**Required JARS ---org.springframework.transaction & aop jars**

Basics

Benefits of Spring Transaction Management

- \* Very easy to use, does not require any underlying transaction API knowledge
- \* Your transaction management code will be independent of the transaction technology
- \* Both annotation- and XML-based configuration
- \* It does not require to run on a server - no server needed

**What is a Transaction?**

A Transaction is a unit of work performed on the database and treated in a reliable way independent of other transaction. In database transaction processing ACID property refers to the Atomicity, Consistency, Isolation, Durability respectively.

**Atomicity**- This property says that all the changes to the data is performed as if they form single operation. For example suppose in a bank application if a fund transfer from one account to another account the atomicity property ensures that if a debit is made successfully in one account the corresponding credit would be made in other account.

**Consistency**- The consistency property of transaction says that the data remains in the consistency state when the transaction starts and ends. for example suppose in the same bank account, the fund transfer from one account to another account, the consistency property ensures that the total value(sum of both account ) value remains the same after the transaction ends.

**Isolation**- This property says that, the intermediate state of transaction are hidden/ invisible to another transaction process.

**Durability**- The Durability says that when the transaction is completed successfully, the changes to the data persist and are not un-done, even in the event of system failure. A transaction is not considered durable until it commits. A system failure entails a database recovery, which includes a rollback procedure for all uncommitted transactions, ultimately leaving the database in a consistent state.

## Transaction Handling

Now, in Java you can handle transactions with plain SQL, with plain JDBC (a bit higher level), using Hibernate (or any other ORM library), or on an even higher level - with EJB or, finally, Spring!

EJBs require an application server, but spring based jdbc application doesn't.

## Ways of Transaction Handling

### Programmatic vs. Declarative

Spring offers two ways of handling transactions: programmatic and declarative. If you are familiar with EJB transaction handling, this corresponds to bean-managed and container-managed transaction management.

Programmatic means you have transaction management code surrounding your business code. That gives you extreme flexibility, but is difficult to maintain and too much of boilerplate code.

Declarative means you separate transaction management from the business code. You only use annotations or XML based configuration.

### As a summary

- \* programmatic management is more flexible during development time but less flexible during application life
- \* declarative management is less flexible during development time but more flexible during application life

## Global transactions Vs Local Transactions

Global transactions enable you to work with multiple transactional resources, typically multiple relational databases . The application server manages global transactions through the JTA. (complex to use through UserTransaction object)

### Local Transactions

Local transactions are resource-specific, such as a transaction associated with a JDBC connection. Local transactions may be easier to use, but have significant disadvantages as they cannot work across multiple transactional resources.

### Spring's solution

Spring resolves the disadvantages of global and local transactions. It enables application developers to use a consistent programming model in any environment. You write your code once, and it can benefit from different transaction management strategies in different environments. It supports both declarative and programmatic transaction management. Most users prefer declarative transaction management.

## API details

1. The key to the Spring transaction abstraction is the notion of a transaction strategy. Its the central interface in Spring's transaction infrastructure. A transaction strategy is defined by the `org.springframework.transaction.PlatformTransactionManager`

interface: which has `TransactionStatus getTransaction(TransactionDefinition td)` throws `TransactionExc`

`TransactionException` --- As in Spring's philosophy, the `TransactionException` that is thrown by any of the `PlatformTransactionManager` interface's methods, is unchecked. Transaction infrastructure failures are generally fatal, managed by spring Tx framework & developer is NOT forced to handle this.

### The `TransactionDefinition` interface specifies:

**Isolation:** The degree to which this transaction is isolated from the work of other transactions.

Concurrent transactions cause problems that might be difficult to investigate.

- \* Lost update - two transactions both update a row, the second transaction aborts, both changes are lost
- \* Dirty read - reading changes that are not yet committed
- \* Unrepeatable read - a transactions reads twice the same row, getting different data each time
- \* Phantom read - similar to the previous one, except that the number of rows changed

Now, the perfect solution to these problems is maximum isolation, but in reality this would cost too much resources and could lead to deadlocks. So, instead, you will set one of five isolation levels (where the fifth one is actually the maximum isolation level):

### Supported levels

**ISOLATION\_DEFAULT** -- Use the default isolation level of the underlying datastore.

**ISOLATION\_READ\_UNCOMMITTED** --- Indicates that dirty reads, non-repeatable reads and phantom reads can occur.

**ISOLATION\_READ\_COMMITTED** --- Indicates that dirty reads are prevented; non-repeatable reads and phantom reads can occur.

**ISOLATION\_REPEATABLE\_READ** -- Indicates that dirty reads and non-repeatable reads are prevented; phantom reads can occur.

**ISOLATION\_SERIALIZABLE** -- Indicates that dirty reads, non-repeatable reads and phantom reads are prevented.

## Transaction Propagation

Whenever a one transactional method calls other transactional method, a decision is made - what to do with the transaction. Create a new one? Use an existing one if it exists, otherwise create a new one? Use an existing one only if it exists, otherwise fail?

### Supported behaviors ---

**MANDATORY** -- Supports a current transaction; throws an exception if no current transaction exists.

**REQUIRED** -- default behavior.

Supports a current transaction; creates a new one if none exists.

**NESTED** ---Executes within a nested transaction if a current transaction exists, otherwise same as REQUIRED

## **SUPPORTS**

Supports a current transaction; executes non-transactionally if none exists.

## **REQUIRES\_NEW**

Creates a new transaction, suspending the current transaction if one exists.

## **NEVER**

Does not support a current transaction; throws an exception if a current transaction exists.

## **NOT\_SUPPORTED**

Does not support a current transaction; always executes non-transactionally.

Timeout: in seconds .How long this transaction runs before timing out and being rolled back automatically by the underlying transaction infrastructure.

Default value = -1 , indefinite w/o time out

Otherwise specify value

eg

**@Transactional(timeout=100)**

**Read-only status:** A read-only transaction can be used when your code reads but does not modify data.

Read-only transactions can be a useful optimization in some cases, such as when you are using Hibernate.

eg -- @Transactional(readOnly = true)

default is false.

## **Rollback behavior**

With Spring transaction management the default behavior for automatic rollback is this: Only unchecked exceptions cause a rollback. Unchecked exceptions are RuntimeExceptions and Errors. But can be changed.

eg --

@Transactional(rollbackFor = IOException.class, noRollbackFor = RuntimeException.class)

public void doSomething(...)

## **Implementation steps & concept for annotation based declarative transaction management.**

1. For plain JDBC implementations of PlatformTransactionManager --- use DataSourceTransactionManager --- implementation class for a single JDBC DataSource.

2. Declare the same in spring configuration xml file. eg --

```
<!-- tx manager bean -->
```

```
<bean id="transactionManager" class="org.springframework.jdbc.datasource.DataSourceTransactionManager"
```

```
p:dataSource-ref="dataSource">
```

```
</bean>
```

3. To enable annotated transaction support , add transaction namespace(tx) & add following

```
<tx:annotation-driven transaction-manager="transactionManager" />
```

Note : can even skip attribute transaction-manager, if id of Tx Mgr bean is transactionManager

This completes configuration steps

4. In service layer beans (typically annotated with @Service) , add method level annotation @Transactional along with suitable properties.

## Hibernate Caching

Caching is a facility provided by ORM frameworks which help users to get fast running web application, while help framework itself to reduce number of queries made to database in a single transaction.

Hibernate achieves this by implementing first level cache.

First level cache in hibernate is enabled by default and you do not need to do anything to get this functionality working. In fact, you can not disable it even forcefully.(typically)

Its associated with Session object.

Session object is created on demand from session factory and it is lost, once the session is closed.

Similarly, first level cache associated with session object is available only till session object is live. It is available to session object only and is not accessible to any other session object in any other part of application.

### NOTE

1. First level cache is associated with session object and other session objects in application can not see it.
2. The scope of cache objects is of session. Once session is closed, cached objects are gone forever.
3. First level cache is enabled by default and you can not disable it.
4. When we query an entity first time, it is retrieved from database and stored in first level cache associated with hibernate session.
5. If we query same object again with same session object, it will be loaded from cache and no sql query will be executed.(completely true only for get or load)
6. The loaded entity can be removed from session using evict(Object ref) method. The next loading of this entity will again make a database call if it has been removed using evict() method.
7. The whole session cache can be removed using clear() method. It will remove all the entities stored in cache.

#####

Caching is facility provided by ORM frameworks which help users to get fast running web application, while help framework itself to reduce number of queries made to database in a single transaction. Hibernate also provide this caching functionality, in two layers.

First level cache: This is enabled by default and works in session scope. Read more about hibernate first level cache.

Second level cache: This is apart from first level cache which is available to be used globally in session factory scope.

Above statement means, second level cache is created in session factory scope and is available to be used in all sessions which are created using that particular session factory.

It also means that once session factory is closed, all cache associated with it die and cache manager also closed down.

## How second level cache works?

1. Whenever hibernate session try to load an entity, the very first place it look for cached copy of entity in first level cache (associated with particular hibernate session).
2. If cached copy of entity is present in first level cache, it is returned as result of load/get method.
3. If there is no cached entity in first level cache, then second level cache is looked up for cached entity.
4. If second level cache has cached entity, it is returned as result of load/get method. But, before returning the entity, it is stored in first level cache also so that next invocation to load method for entity will return the entity from first level cache itself, and there will not be need to go to second level cache again.

If entity is not found in first level cache and second level cache also, then database query is executed and entity is stored in both cache levels, before returning as response of load() method.

=====

## Spring Web Hibernate Integration

1. Create dynamic web project
2. Create User library --spring-hibernate-rest jars  
DO NOT add any other library.
3. Add DispatcherServlet entry in web.xml -- to ensure all request pass through central dispatcher servlet.
4. Create spring-servlet.xml under <WEB-INF> -- To allow D.S to create Web application context using master config xml file.
  - 4.1 Copy earlier entries.(ctx,mvc & view resolver)
5. Create <resources> & copy database.properties & hibernate-persistence.xml from <spring-hibernate-templates>

### What it contains ---

- 5.1 DataSource bean --- Apache (Connection pool)
- 5.2 SF bean -- Spring
- 5.3 Tx Mgr --- Spring
- 5.4 enabled anno support for Txs(@Transactional)
- 6 import hibernate-persistence.xml into spring-servlet.xml

Configuration steps over....

7. Identify persistence requirements & create POJO/Model/DTO.

POJO properites --- represent 1. DB cols 2.Request params --i.e clnt's conversational state.

+ P.L validation rules --anno.

class level --@Entity,@Table

Anno -- field level --- @NotEmpty,@NotNull,@Email....

Annotation -- prop level(getter) --@Id,@Column....

8. Create DAO layer

I/F -- Dao i/f --- validateCustomer

Implementation class ---

dependency --- SessionFactory -- @AutoWired

No need to manage TxS --directly get session from SF & perform CRUD operation.

9. Create Service Layer --i/f & then implementation class

@Service & @Transactional --- annotations.

Inject dependency of Dao Layer.

10 Create or copy existing controllers & test the flow.

## =====

## Hibernate Inheritance

**Inheritance is one of the fundamental design principles in OOP. BUT relational databases don't support inheritance**

JPA suggests different strategies to support inheritance hierarchies.

### Hibernate Inheritance strategies

#### 1 Single Table Strategy

In this inheritance, a single table is used to store all the instances of the entire inheritance hierarchy. The Table will have a column for every attribute of every class in the hierarchy. Discriminator columns identifies which class a particular row belongs.

Annotations used in super class

@Entity

@Inheritance(strategy = InheritanceType.SINGLE\_TABLE)

@DiscriminatorColumn(name = "emp\_type")

@Table(name = "employees")

public class Employee {...}

In sub class :

@Entity

@DiscriminatorValue("worker")

public class Worker extends Employee {...}

@Entity

@DiscriminatorValue("mgr")

public class Manager extends Employee {...}

With this mapping strategy, only a single table will be created for both concrete classes (Manager and Worker).

Hibernate will create a discriminator column named emp\_type to differentiate each concrete type. The value of this column will be either worker or mgr

#### 2. Joined Table Strategy

This is the most logical solution, as it mirrors the object structure in the database. In this approach, a separate database table is defined for each of the class in the hierarchy and each table stores only its local attributes.

```
@Entity
```

```
@Inheritance(strategy = InheritanceType.JOINED)
```

```
public class Product {  
    @Id @GeneratedValue  
    private Long id;  
    private String name;  
}
```

```
@Entity
```

```
public class Book extends Product {  
    private String isbn;  
}
```

The above mapping will create two tables, one for super class Product and another for entity Book. Both the tables will have Product id column. The primary key of table Book will have foreign key relationship with primary key of Product table.

### 3. Table Per class

Not supported by all JPA vendors .

A separate table is defined for each concrete class in the inheritance hierarchy to store all the attribute of that class and all its super classes.

### 4. @MappedSuperClass

We want to extract common behavior in a super class in JPA entities but without having a table for that super class. How would you achieve that?

If we create a normal class as the super class, then as per JPA specifications, the fields for that class are not persisted in the database tables. We need to create a super class extracting the common fields and then annotate that class with @MappedSuperClass in order to persist the fields of that super class in subclass tables. A mapped super class has no separate table defined for it.

MappedSuperClass example

```
@MappedSuperclass
```

```
public class Person {  
    @Id  
    private Long id;  
    private String name;  
}
```

```
@Entity
```

```
public class Employee extends Person {  
    private String company;  
}
```

The above configuration will result in a single table for employee with 3 columns (id, name and company), 1 from Employee class itself and 2 are inherited from Person class. Person class will never have its own table in this case.

## Spring-MVC Request Flow

1. Thin client sends request along with request parameters (\*.htm)
2. First it reaches Spring's DispatcherServlet which acts like front controller in MVC model II. It is a common web-application pattern where a single servlet delegates responsibility for a request to other components of an application to perform the actual processing.
3. The DispatcherServlet's job is to send the request on to a Spring MVC controller (program supplied)
4. As typical application may have several controllers, DispatcherServlet consults Handler mapping to select controller.
5. The Handler mapping will choose a controller based on request URL.
6. Request thus reaches the controller. Controller may use one or more service layer objects for execution of business logic.

The result of business logic needs to be carried back to the user and displayed in the browser. This information is the model. It has to be sent to JSP (or any other view template) for converting it to HTML like format.

7. So the controller will package up the model data and the name of a view into a ModelAndView object. It returns the request+ ModelAndView back to the DispatcherServlet. (Model and View)

doesn't carry a reference to the actual JSP but only carries a logical result name that will be used to look up the actual view that will produce the resulting HTML.

8. DispatcherServlet now consults a view resolver to find the actual JSP. & delivers the model data to view JSP
9. The view will use the model data to render a page that will be sent as dynamic response back to the client browser object.

The ViewResolver provides a mapping between view names and actual views.

UrlBasedViewResolver Simple implementation of the ViewResolver interface that effects the direct resolution of logical view names to URLs, without an explicit mapping definition. This is appropriate if your logical names match the names of your view resources in a straightforward manner, without the need for arbitrary mappings.

## Spring Boot

### Problems with traditional/legacy spring

We use different modules from spring such as core module, to do dependency injection. The MVC module to develop the web layer for our application or even the restful web services layer. And then the DAO layer where we use the spring JDBC/ORM which makes our life easy to develop a data access layer for our application. When we are using ORM tools like Hibernate, we can use spring data JPA and we use these modules and more that are valuable from Spring.

Initially we used XML based configuration or annotations based configuration, This configuration can get difficult and hard to maintain over time. And also we need to make sure that each of these modules is available for our application by defining all the dependencies in the Maven pom.xml. And at runtime we have to be sure that these versions of various Modules that we use are compatible with each other. But it's our responsibility to do all that, and once we have all that in place we will build our application and will have to deploy to an external web container to test it.

**Spring boot will automate all this for us.**

### **What are Spring Boot Features ?**

1. The first of those super cool features is auto configuration - Spring Boot automatically configures everything that is required for our application. We don't have to use XML or annotation based or Java configuration anymore.

For example if you are using Spring MVC or the web to develop a web application or a restful web service application spring boot will automatically configure the dispatcher servlet and does all the request mapping for us. We don't have to use any xml or annotation based configuration to configure this servlet.

Similarly if you are using spring data or object relational mapping while working with tools like Hibernate to perform database crud we no longer have to configure the data source or even the transaction manager. Spring boot will automatically configure these for our application.

2. The next super cool feature is spring boot starters. With the spring boot starters spring boot takes the problem of module availability we need. Before Spring Boot we had to make sure that a particular library required for our project is available and also the versions of different libraries are compatible. But we don't have to do that anymore thanks to spring boot starters every spring boot project will have a parent project. This project has all the version information of various libraries that we will be using in our project so we need not worry about version compatibility. The spring developers have already done it for us and put all that information into this spring boot starter parent.

Secondly we have starters for different types of projects if we are developing a web project then we simply need to include the starter web. We don't have to include any other libraries or dependencies. Spring boot will automatically pull all the other libraries that are required because this project here or this or dependency transitively depends on other libraries that are required. Maven will automatically pull those libraries. The spring boot developers have given us starter dependencies which when we use in our projects will not have the Modular availability problem and the version compatibility problem.

Another example is the spring boot starter data.jpa. When you want to work with Hibernate you simply include the single dependency in your maven pom.xml and all the other libraries will be pulled accordingly. And also the correct versions of those libraries will be included because all that version information is available in this spring boot starter parent which is a parent for every spring boot project.

3. The third super cool feature we don't have to worry about deploying our applications to external container spring boot comes with an embedded servlet container and these containers.

By default it is Tomcat but you can also use Jetty and undertow or any other external server. So no longer external deployment you can simply right click on your project and run it and your application will be launched on its embedded Tomcat server by default.

4. Last and very important spring boot gives us a lot of health checks of our application for free through the spring boot actuators. We can use different types of health checks that come for free and we can use these even on production when our application is running. These can be activated easily and will display all the auto configuration reports and everything that is automatically configured for our application.

### **Important components of a Spring Boot Application**

Below is the starting point of a Spring Boot Application

**@SpringBootApplication**

```
public class HellospringbootApplication { p.s.v.m(...) {...}}
```

@SpringBootApplication - This is where all the spring boot magic happens.

The SpringBootApplication is a key annotation which is a top level annotation which contains several other annotations on it.

@SpringBootConfiguration

@EnableAutoConfiguration

@ComponentScan

The first one @SpringBootConfiguration tells spring boot or the container that this class here can have several bean definitions. We can define various spring beans here and those beans will be available at run time so you define a method here .

The second annotation @EnableAutoConfiguration is a very important annotation at enable Auto configuration. This annotation tells spring boot to automatically configure the spring application based on the dependencies that it sees on the classpath.

eg:

If we have a MySQL dependency in our pom.xml as automatically Spring Boot will create a data source. We will also provide other information like username password etc. but spring boot will scan through all these dependencies and it will automatically configure the data source required for us.

Another example is spring web, If we have spring web in your dependencies.

Then spring boot will automatically create the dispatcher servlet on all that configuration file you for free.

All the xml, all the java based configuration is now gone. We as developers need not do all that configuration it all comes for free thanks to spring boots to enable auto configuration annotation.

### @ComponentScan

So this tells us that spring boot or spring should scan through the classes and see which all classes are marked with the stereotype annotations like @Component Or @Service @Repository and manage these spring beans . Default base-pkg is the pkg in which main class is defined.

Can be overridden by

eg :

```
@ComponentScan(basePackages = "com")
```

For scanning entities :

```
@EntityScan(basePackages = "com.app.pojos")
```

## Enter Spring boot

### 1. What is Spring Boot?

Spring Boot is a Framework from "The Spring Team" to ease the bootstrapping and development of new Spring Applications.

It provides defaults for code and annotation configuration to quick-start new Spring projects within no time.

It follows "Opinionated Defaults Configuration" an approach to avoid lot of boilerplate code and configuration to improve Development, Unit Test and Integration Test Process.

## 2. What is NOT Spring Boot?

Spring Boot Framework is not implemented from the scratch by The Spring Team

It's implemented on top of existing Spring Framework (Spring IO Platform).

It is not used for solving any new problems. It is used to solve same problems like Spring Framework.

(i.e to help in writing enterprise applications)

## 3. Advantages of Spring Boot:

It is very easy to develop Spring Based applications with Java

It reduces lots of development time and increases productivity.

It avoids writing lots of boilerplate Code, Annotations and XML Configuration.

It is very easy to integrate Spring Boot Application with its Spring Ecosystem like Spring JDBC, Spring ORM, Spring Data, Spring Security etc.

It follows "Opinionated Defaults Configuration" Approach to reduce Developer effort

It provides Embedded HTTP servers like Tomcat, Jetty etc. to develop and test our web applications very easily.

It provides CLI (Command Line Interface) tool to develop and test Spring Boot(Java or Groovy)

Applications from command prompt very easily and quickly.

It provides lots of plugins to develop and test Spring Boot Applications very easily using Build Tools like Maven and Gradle

It provides lots of plugins to work with embedded and in-memory Databases very easily.

In short

Spring Boot = Spring Framework + Embedded HTTP Server(eg Tomcat) - XML Based configuration - efforts in identifying dependencies in pom.xml

## 4. What is that "Opinionated Defaults Configuration" ?

When we use Hibernate/JPA, we would need to configure a datasource, a session factory, a transaction manager among lot of other things.

Refer to our hibernate-persistence.xml , to recall what we did earlier .

### Spring Boot says can we look at it differently ?

Can we auto-configure a Data Source(connection pool) / session factory / Tx manager if Hibernate jar is on the classpath?

**It says :**

When a spring mvc jar is added into an application, can we auto configure some beans automatically?

(eg HandlerMapping , ViewResolver n configure DispatcherServlet)

**By the way :**

There would be of course provisions to override the default auto configuration.

## 5. How does it work ?

Spring Boot looks at

1. Frameworks available on the CLASSPATH

## 2. Existing configuration for the application.

Based on these, Spring Boot provides basic configuration needed to configure the application with these frameworks. This is called Auto Configuration.

## 6. What is Spring Boot Starter ?

Starters are a set of convenient dependency descriptors that you can include in your application's pom.xml

.

eg : Suppose you want to develop a web application.

W/o Spring boot , we would need to identify the frameworks we want to use, which versions of frameworks to use and how to connect them together.

BUT all web application have similar needs.

These include Spring MVC, Jackson Databind (for data binding), Hibernate-Validator (for server side validation using Java Validation API) and Log4j (for logging). Earlier while creating any web app, we had to choose the compatible versions of all these frameworks.

With Spring boot : You just add Spring Boot Starter Web.

Dependency for Spring Boot Starter Web

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

Just by adding above starter , it will add lot of JARs under maven dependencies

Another eg : If you want to use Spring and JPA for database access, just include the spring-boot-starter-data-jpa dependency in your project, and you are good to go.

7. Another cool feature of Spring boot is : we don't have to worry about deploying our applications to external container. It comes with an embedded servlet container.

## 8. Important components of a Spring Boot Application

Below is the starting point of a Spring Boot Application

### @SpringBootApplication

```
public class HelloSpringBootApplication {
    public static void main(String[] args) {
        SpringApplication.run(HelloSpringBootApplication.class, args);
    }
}
```

About : org.springframework.boot.SpringApplication

It's Class used to bootstrap and launch a Spring application from a Java main method.

By default class will perform the following steps to bootstrap the application

1. Create an ApplicationContext instance (representing SC)
2. Manages life cycle of spring beans

@SpringBootApplication - This is where all the spring boot magic happens.

It consists of following 3 annotations.

### 1. @SpringBootConfiguration

It tells spring boot that this class here can have several bean definitions. We can define various spring beans here and those beans will be available at run time .

### 2. @EnableAutoConfiguration

It tells spring boot to automatically configure the spring application based on the dependencies that it sees on the classpath.

eg:

If we have a MySQL dependency in our pom.xml , Spring Boot will automatically create a data source,using the properties in application.properties file.

If we have spring web in pom.xml , then spring boot will automatically create the dispatcher servlet n other beans (HandlerMapping , ViewResolver)

All the xml, all the java based configuration is now gone.It all comes for free thanks to spring boot to enable auto configuration annotation.

### 3. @ComponentScan (equivalent to xml tag : context:component-scan)

So this tells us that spring boot to scan through the classes and see which all classes are marked with the stereotype annotations like @Component Or @Service @Repository and manage these spring beans . Default base-pkg is the pkg in which main class is defined.

Can be overridden by

eg :

```
@ComponentScan(basePackages = "com")
```

For scanning entities : (equivalent to packagesToScan)

```
@EntityScan(basePackages = "com.app.pojos")
```

Steps

1. File --New --Spring starter project -- add project name , group id ,artifact id ,pkg names , keep packaging as JAR for Spring MVC web application.

2. Add dependencies

web -- web

sql -- Spring Data JPA, MYSQL

Core -- DevTools

Lombok

validation

3. Copy the entries from supplied application.properties & edit DB configuration

4. For Spring MVC (with JSP view layer demo) using spring boot project

Add following dependencies ONLY for Spring MVC with JSP as View Layer in pom.xml

```
<!-- Additional dependencies for Spring MVC -->

<dependency>

    <groupId>org.apache.tomcat.embed</groupId>

    <artifactId>tomcat-embed-jasper</artifactId>

</dependency>

<dependency>

    <groupId>javax.servlet</groupId>

    <artifactId>jstl</artifactId>

</dependency>
```

5. Create under src/main/webapp : WEB-INF folder

6. Create test it.

7. Port earlier spring MVC app , observe the problems.

& fix it.

Problem observed : ??????

**Reason :** Could not find org.hibernate.SessionFactory (since Spring boot DOES NOT support any native hibernate implementation directly)

**Solution :** Replace hibernate's native API (org.hibernate) by JPA

(refer : day17-data\day17\_help\diagrams\jpa-entitymgr-session-layers.png)

In DAO layer : replace native hibernate API by JPA

i.e instead of auto wiring SF in DAO layer : inject JPA' EntityManager directly in DAO.

**How ?**

@PersistenceContext

//OR can continue to use @AutoWired : spring supplied annotation

private EntityManager mgr;

//uses def persistence unit , created auto by spring boot using db setting added //in application.properties file , 1 per app / DB

Use directly EntityManager API (refer : )

OR

Unwrap hibernate Session from EntityManager

Session session = mgr.unwrap(Session.class);

Which one is preferred ? 1st soln.

8. Test Entire application.

---

## Spring boot internals explained :

### Important components of a Spring Boot Application

Below is the starting point of a Spring Boot Application

@SpringBootApplication

```
public class HelloSpringBootApplication {  
  
    public static void main(String[] args) {  
        SpringApplication.run(HelloSpringBootApplication.class, args);  
    }  
}
```

About : org.springframework.boot.SpringApplication

It's Class used to bootstrap and launch a Spring application from a Java main method.

By default class will perform the following steps to bootstrap the application

1. Create an ApplicationContext instance (representing SC) , based upon default configuration.
2. Manages life cycle of spring beans
3. Launches embedded Tomcat container

**@SpringBootApplication** - This is where all the spring boot magic happens.

It consists of following 3 annotations.

#### 1. @SpringBootConfiguration

It tells spring boot that this class here can have several bean definitions. We can define various spring beans here and those beans will be available at run time .

eg : ModelMapper (used for mapping Entity <----> DTO) , PasswordEncoder

#### 2. @EnableAutoConfiguration

It tells spring boot to automatically configure the spring application based on the dependencies that it sees on the classpath.

eg:

If we have a MySQL dependency in our pom.xml , Spring Boot will automatically create a data source(Connection Pool) ,using the properties in application.properties file.

If we have spring web in pom.xml , then spring boot will automatically create the dispatcher servlet n other beans (HandlerMapping , ViewResolver)

All the xml, all the java based configuration is now gone.It all comes for free thanks to spring boot to enable auto configuration annotation.

#### 3. @ComponentScan (equivalent to xml tag : context:component-scan)

Default base-pkg is the pkg in which main class is defined.

So this tells us that spring boot to scan through the classes and see which all classes are marked with the stereotype annotations like @Component Or @Service @Repository and manage these spring beans .

Can be overridden by

eg :

```
@ComponentScan(basePackages = "com")
```

For scanning entities : (equivalent to packagesToScan)

```
@EntityScan(basePackages = "com.app.pojos")
```

=====

## Regarding Form Binding

1. If there multiple request params(use case -- register/update) --- bind POJO directly to a form.(2 way form binding technique)

### How ?

1.1 For loading the form (in showForm method of the controller) , bind empty POJO (using def constr) in model map

### How ?

Explicit --add Model as dependency & u add

```
map.addAttribute(nm,val)
```

OR better way

implicit -- add POJO as a dependency

eg : User registration

```
@GetMapping("/reg")
```

```
public String showForm(User u) {...}
```

### What will SC do ?

```
SC --- User u=new User();
```

chks --- Are there any req params coming from client ? --- typically --no --- only getters will be called --

adds pojo as model attr (in Model map)

```
map.addAttribute("user",new User());
```

1.2 In form (view ---jsp) -use spring form tags along with modelAttribute

Steps

1. import spring supplied form tag lib

2. Specify the name of modelAttribute under which form data will be exposed.(name of model attr mentioned in the controller)

```
<s:form method="post" modelAttribute="user">
```

```
<s:input path="email"/>.....
```

```
</s:form>
```

1.3 Upon form submission (clnt pull I)

clnt sends a new req --- containing req params

```
@PostMapping("/reg")
```

```
public String processForm(User u,RedirectAttributes flashMap,HttpSession hs) {
```

```
//SC calls
```

```
User u=new User();
```

SC invokes MATCHING (req param names --POJO prop setters)

setters. -- conversational state is transferred to Controller.

adds pojo as model attr (in Model map)

```
map.addAttribute("user",u)
```

Thus you get a populated POJO directly in controller w/o calling <jsp:setProperty> & w/o using any java bean.

=====

## Maven

### What is Maven ?

Build automation tool for overall project management.

It helps in

1. checking a build status
2. generating reports (basically javadocs)
3. setting up the automated build process and monitors the same.

### Why Maven ?

It eases out source code compilation, distribution, documentation, collaboration with different teams .

Maven tries 2 describe

1. How a software is built.
2. The dependencies, plug-ins & profiles that the project is associated in a standalone or a distributed environment.

Vendor -- Apache

Earlier build tool -- Ant

Vendor -- Apache.

### Ant disadvantages

1. While using ant , project structure had to be defined in build.xml. Maven has a convention to place source code, compiled code etc. So no need to provide information about the project structure in pom.xml file.
2. Maven is declarative, everything you define in the pom.xml file.No such support in ant.
3. There is no life cycle in Ant, where as life cycle exists in Maven.

### Maven advantages

4. Managing dependencies
5. Uses Convention over configuration - configuration is very minimal

6. Multiple/Repeated builds can be achieved.
7. Plugin management.
8. Testing - ability to run JUnit and other integration test suites.

### What is POM? (Project Object Model)

It is the core element of any maven project.

Any maven project consists of one configuration file called pom.xml.

Location --In the root directory of any maven project.

It contains the details of the build life cycle of a project.

Contents

Dependencies used in the projects (Jar files)

Plugins used

Project version

Developers involved in the project

Build profiles etc.

Maven reads the pom.xml file, then executes the goal.

Elements of maven pom.xml file

=====

### Maven Steps

1. project        It is the root element of pom.xml file.
2. modelVersion        It is the sub element of project. It specifies the modelVersion.
3. groupId        It is the sub element of project. It specifies the id for the project group.(typically organization name)
4. artifactId        It is the sub element of project. It specifies the id for the artifact (project). An artifact is something that is either produced or used by a project. Examples of artifacts produced by Maven for a project include: JARs, WARs.
5. version        It is the sub element of project. It specifies the version of the artifact under given group.
6. packaging -- defines packaging type such as jar, war etc.
7. name --        defines name of the maven project.
8. plugins        ---compiler plugins , eclipse plugins
9. dependencies -- collection of dependencies for this project.

Within that --

dependency -- defines a specific dependency.(eg : hibernate dependency,spring web)

9. scope --        defines scope for this maven project. It can be compile, provided, runtime, test and system.

### Goals in Maven

Goal in maven is nothing but a particular task which leads to the compiling, building and managing of a project. A goal in maven can be associated to zero or more build phases. Only thing that matters is the order of the goals

defined for a given project in pom.xml. Because, the order of execution is completely dependent on the order of the goals defined.

eg : clean , build ,install ,test

### **What is a Maven Repository**

A maven repository is a directory of packaged JAR file with pom.xml file. Maven searches for dependencies(JARs) in the repositories. There are 3 types of maven repository:

#### **Local Repository**

#### **Central Repository**

#### **Remote Repository**

Maven searches for the dependencies in the following order:

Local repository then Central repository then Remote repository.

maven repositories

If dependency is not found in these repositories, maven stops processing and throws an error.

### **1. Maven Local Repository**

Maven local repository is located in the file local system. It is created by the maven when you run any maven command.

By default, maven local repository is HOME / .m2 directory.

(Can be updated by changing the MAVEN\_HOME/conf/settings.xml)

### **2) Maven Central Repository**

Maven central repository is located on the web(Created by the apache maven community)

The path of central repository is: <https://mvnrepository.com/repos/central>

### **3) Maven Remote Repository**

Maven remote repository is also located on the web. Some of libraries that are missing from the central repository eg JBoss library , Oracle driver etc, can be located from remote repository.

### **Maven Build Life Cycle**

#### **What is it ?**

The sequence of steps which is defined in order to execute the tasks and goals of any maven project is known as build life cycle in maven.

Maven comes with 3 built-in build life cycles

**Clean** - this phase involves cleaning of the project (for a fresh build & deployment)

**Default** - this phase handles the complete deployment of the project

**Site** - this phase handles the generating the java documentation of the project.

Build Profiles in Maven

It is a subset of elements which allows to customize builds for particular environment. Profiles are also portable for different build environments.

Build environment basically means a specific environment set for production and development instances. When developers work on development phase, they use test database from the production instance and for the production phase, the live database will be used.

So, in order to configure these instances maven provides the feature of build profiles. Any no. of build profiles can be configured and also can override any other settings in the pom.xml

eg : profiles can be set for dev, test and production phases.

Installation (w/o IDE)

1. Download Maven from Apache (version 3.x)
2. Add MAVEN\_HOME as environment variable
3. Add maven/bin under path (for easy accessibility)
4. Verify maven

```
mvn --version
```

OR use m2e plug-in (a standard part of Eclipse for J2EE)

---

## REST

### Background

Web services have really come a long way since its inception. In 2002, the World Wide Web consortium(W3C) had released the definition of WSDL(web service definition language) and SOAP web services. This formed the standard of how web services are implemented.

In 2004, the web consortium also released the definition of an additional standard called RESTful. Over the past couple of years, this standard has become quite popular. And is being used by many of the popular websites around the world which include Facebook and Twitter.

REST is a way to access resources which lie in a particular environment. For example, you could have a server that could be hosting important documents or pictures or videos. All of these are an example of resources. If a client, say a web browser needs any of these resources, it has to send a request to the server to access these resources. Now REST defines a way on how these resources can be accessed.

eg of a web application which has a requirement to talk to other applications such Facebook, Twitter, and Google.

Now if a client application had to work with sites such as Facebook, Twitter, etc. they would probably have to know what is the language Facebook, Google and Twitter are built on, and also on what platform they are built on.

Based on this, we can write the interfacing code for our web application, but this could prove to be a nightmare.

So instead , Facebook, Twitter, and Google expose their functionality in the form of Restful web services. This allows any client application to call these web services via REST.

refer to diag --REST 0.png

### What is Restful Web Service?

REST is used to build Web services that are lightweight, maintainable, and scalable in nature. A service which is built on the REST architecture is called a RESTful service. The underlying protocol for REST is HTTP, which is the basic web protocol. REST stands for REpresentational State Transfer

The key elements of a RESTful implementation are as follows:

1. **Resources** – The first key element is the resource itself. Let assume that a web application on a server has records of several employees. Let's assume the URL of the web application is `http://www.server.com`. Now in order to access an employee record resource via REST, one can issue the command `http://www.server.com/employee/1` - This command tells the web server to please provide the details of the employee whose employee number is 1.
2. **Request Verbs** - These describe what you want to do with the resource. A browser issues a GET verb to instruct the endpoint it wants to get data. However, there are many other verbs available including things like POST, PUT, and DELETE. So in the case of the example `http://www.server.com/employee/1`, the web browser is actually issuing a GET Verb because it wants to get the details of the employee record.
3. **Request Headers** – These are additional instructions sent with the request. These might define the type of response required or the authorization details.
4. **Request Body** - Data is sent with the request. Data is normally sent in the request when a POST request is made to the REST web service. In a POST call, the client actually tells the web service that it wants to add a resource to the server. Hence, the request body would have the details of the resource which is required to be added to the server.
5. **Response Body** – This is the main body of the response. So in our example, if we were to query the web server via the request `http://www.server.com/employee/1`, the web server might return an XML document with all the details of the employee in the Response Body.
6. **Response Status codes** – These codes are the general codes which are returned along with the response from the web server. An example is the code 200 which is normally returned if there is no error when returning a response to the client.

## Restful Methods

The below diagram shows mostly all the verbs (POST, GET, PUT, and DELETE) and an example of what they would mean.

Refer to -- REST 1.png

Let's assume that we have a RESTful web service is defined at the location. `http://www.server.com/employee`. When the client makes any request to this web service, it can specify any of the normal HTTP verbs of GET, POST, DELETE and PUT. Below is what would happen if the respective verbs were sent by the client.

**POST** – This would be used to create a new employee using the RESTful web service

**GET** - This would be used to get a list of all employee using the RESTful web service

**PUT** - This would be used to update all employee using the RESTful web service

**DELETE** - This would be used to delete all employee using the RESTful web service

=====

## What is REST ?

REST stands for REpresentational State Transfer.

### Why the name ?

It's the representation of the resource (eg : image/student/customer/invoice/bill....) using known data exchange formats(text/xml/json) --which gets transferred between client & server .

REST is web standards based architecture and uses HTTP Protocol for data communication.

It revolves around resource where every component is a resource and a resource is accessed by a common interface using HTTP standard methods. (ROA)

REST was first introduced by Roy Fielding in 2000.

In REST architecture, a REST Server simply provides access to resources and REST client accesses and presents the resources.

Here each resource is identified by URIs

REST uses various representations to represent a resource like text, JSON and XML. Most popular light weight data exchange format used in web services = JSON

### HTTP Methods

Following well known HTTP methods are commonly used in REST based architecture.

GET - Provides a read only access to a resource.

POST - Used to create a new resource.

DELETE - Used to remove a resource.

PUT - Used to update a existing resource or create a new resource.

### RESTful Web Services

A web service is a collection of open protocols and standards used for exchanging data between applications or systems.

Platform & technology independent solution.

Software applications written in various programming languages and running on various platforms can use web services to exchange data over computer networks like the Internet in a manner similar to inter-process communication on a single computer.

This interoperability (e.g., between Java and Python, or Windows and Linux applications or java & .net ) is due to the use of open standards.

Web services based on REST Architecture are known as RESTful web services.

These web services use HTTP methods to implement the concept of REST architecture. A RESTful web service usually defines a URI, Uniform Resource Identifier a service, provides resource representation such as JSON and set of HTTP Methods.

---

### Annotations

1. **@PathVariable** --- handles URL templates.

eg : URL -- http://host:port/test\_web/products/10

Method of Controller

**@GetMapping("/{pid}")**

```
public Product getDetails(@PathVariable int pid)
```

```
{...}
```

In the above URL , the path variable {pid} is mapped to an int . Therefore all of the URIs such as /products/1 or /products/10 will map to the same method in the controller.

2. The **@ResponseBody** annotation is used to marshall(serialize) the return value into the HTTP response body. Spring comes with converters that convert the Java object into a format understandable for a client.

3.The **@RequestBody** annotation, instead, unmarshalls the HTTP request body into a Java object injected in the method.

---

**The RestTemplate** is similar to other Spring templates such as JmsTemplate and JdbcTemplate in that Spring eliminates a lot of boot strap code and thus makes your code much cleaner. When applications use the RestTemplate they do not need to worry about HTTP connections, that is all encapsulated by the template. They also get a range of APIs from the RestTemplate which correspond to the well know HTTP methods (GET, PUT, POST, DELETE, HEAD, OPTIONS). These APIs are overloaded to cater for things like different ways of passing parameters to the actual REST API.

---

### Create restful service provider project & then develop its client

For spring restful web service --actually only spring web mvc , hibernate validator & json jars are sufficient.

#### Server side steps

1. Create Spring Web MVC application
2. The layers --service --dao--pojo --DB are the same.
3. In controller layer , replace @Controller by @RestController annotation, at class level.
4. Request Handling methods will respond to different HTTP methods  
(get/post/put/delete)
5. For method=get

Can directly return either a resource eg : Person,BankAccount or Customer or still better is can return entire HTTP response encapsulated in ResponseEntity<T>

What is org.springframework.http.ResponseEntity<T>

Represents HTTP response entity, consisting of status,headers and body.

#### Constructors

1. public ResponseEntity(T body,HttpStatus statusCode)

Create a new ResponseEntity with the given body and status code, and no headers.

2. public ResponseEntity(T body,MultiValueMap<String,String> headers, HttpStatus statusCode)

Create a new ResponseEntity with the given body and status code, and headers.

1. In the controller's get handling method

- 1.1 In @RequestMapping(value = "add uri with template variables")

Return type of the method = either a resource or resource embedded in the ResponseEntity.

2. Use service layer to fetch a resource.

3. Return it to the client.

eg : @GetMapping(value="/cust/{id}")

```
public Customer getCustomerById(@PathVariable int id) {  
    invoke service layer method to get customer details  
}
```

2. In the controller's post handling method

1. In `@PostMapping(value = "uri")`

Add `@RequestBody` annotated method argument

Return type of the method = either a resource or resource embedded in the `ResponseEntity`.

3. Use service layer to fetch a resource.

3. Return it to the client.

---

**@RestController** : Spring 4's new `@RestController` annotation.

Its a combination of `@Controller` and `@ResponseBody`.

**@RequestBody** : If a method parameter is annotated with `@RequestBody`, Spring will bind the incoming HTTP request body(for the URL mentioned in `@RequestMapping` for that method) to that parameter. While doing that, Spring will [behind the scenes] use HTTP Message converters to convert the HTTP request body into domain object [deserialize request body to domain object], based on `ACCEPT` or `Content-Type` header present in request.

**@ResponseBody** : If a method is annotated with `@ResponseBody`, Spring will bind the return value to outgoing HTTP response body. While doing that, Spring will [behind the scenes] use HTTP Message converters to convert the return value to HTTP response body [serialize the object to response body], based on `Content-Type` present in request HTTP header. As already mentioned, in Spring 4, no need to use this annotation.

**@ResponseEntity** is a real deal. It represents the entire HTTP response. Good thing about it is that you can control anything that goes into it. You can specify status code, headers, and body. It comes with several constructors to carry the information you want to sent in HTTP Response.

**@PathVariable** This annotation indicates that a method parameter should be bound to a URI template variable [the one in '{'}].(binding between request handling method parametere & URI template variable)

**MediaType** : With `@RequestMapping` annotation, you can additionally, specify the `MediaType` to be produced or consumed (using `produces` or `consumes` attributes) by that particular controller method, to further narrow down the mapping.

---

## URI Template Patterns

URI templates can be used for convenient access to selected parts of a URL in a `@RequestMapping` method.

A URI Template is a URI-like string, containing one or more variable names. When you substitute values for these variables, the template becomes a URI. The proposed RFC for URI Templates defines how a URI is parameterized. For example, the URI Template `http://www.example.com/users/{userId}` contains the variable `userId`. Assigning the value `fred` to the variable yields `http://www.example.com/users/fred`.

In Spring MVC you can use the `@PathVariable` annotation on a method argument to bind it to the value of a URI template variable:

```
@RequestMapping(value="/owners/{ownerId}", method=RequestMethod.GET)
```

```

public String findOwner(@PathVariable String ownerId, Model model) {
    Owner owner = ownerService.findOwner(ownerId);
    model.addAttribute("owner", owner);
    return "displayOwner";
}

```

The URI Template `/owners/{ownerId}` specifies the variable name `ownerId`. When the controller handles this request, the value of `ownerId` is set to the value found in the appropriate part of the URI.

For example, when a request comes in for `/owners/abc`, the value of `ownerId` is `abc`.

To process the `@PathVariable` annotation, Spring MVC needs to find the matching URI template variable by name. You can specify it in the annotation:

```

@RequestMapping(value="/owners/{ownerId}", method=RequestMethod.GET)
public String findOwner(@PathVariable("ownerId") String theOwner, Model model) {
    // some implementation
}

```

Or if the URI template variable name matches the method argument name you can omit that

detail. As long as your code is not compiled without debugging information, Spring MVC will match the method argument name to the URI template variable name

---

## Rest Controller Vs MVC

A key difference between a traditional MVC controller and the RESTful web service controller is the way that the HTTP response body is created. Rather than relying on a view technology to perform server-side rendering of the data to HTML, typically a RESTful web service controller simply populates and returns a java object. The object data will be written directly to the HTTP response as JSON/XML/Text

To do this, the `@ResponseBody` annotation on the return type of the request handling method tells Spring MVC that it does not need to render the java object through a server-side view layer, but that instead the java object returned is the response body, and should be written out directly.

The java object must be converted to JSON. Thanks to Spring's HTTP message converter support, you don't need to do this conversion manually. Because Jackson Jar is on the classpath, SC can automatically convert the java object to JSON & vice versa (using 2 annotations `@ResponseBody` & `@RequestBody`)

### API --Starting point

`o.s.http.converter.HttpMessageConverter<T>`

--Interface that specifies a converter that can convert from and to HTTP requests and responses.

T --type of request/response body.

### Implementation classes

1. `o.s.http.converter.xml.Jaxb2RootElementHttpMessageConverter`

-- Implementation of `HttpMessageConverter` that can read and write XML using JAXB2.(Java architecture for XML binding)

## 2. o.s.http.converter.json.

`MappingJackson2HttpMessageConverter`

--Implementation of `HttpMessageConverter` that can read and write JSON using Jackson 2.x's `ObjectMapper` class API

### Annotations

Good news is `@RestController` = `@Controller`(at the class level) + `@ResponseBody` added on ret types of all request handling methods

`@ResponseBody` ---marshalling the response.

marshalling (serialization) -- java object --> xml/JSON

`@RequestBody` annotation must be still added on a method argument of request handling method , for unmarshaling(de serialization) though.

`@PathVariable` --- handles URL templates.

eg : URL -- `http://host:port/test_web/products/10`

Method of Controller

`@GetMapping("/{pid}")`

`public Product getDetails(@PathVariable int pid)`

`{...}`

In the above URL , the path variable `{pid}` is mapped to an int . Therefore all of the URIs such as `/products/1` or `/products/10` will map to the same method in the controller.

### • Optimistic Locking

When we talk about locking we are often referring to optimistic locking. The optimistic locking model subscribes to the philosophy that there is a good chance that the transaction in which changes are made to an entity will be the only one that actually changes the entity during that interval. This translates into the decision to not acquire a lock on the entity until the change is actually made to the database, usually at the end of the transaction.

When the data actually does get sent to the database to get updated at flush time or at the end of the transaction, the entity lock is acquired and a check is made on the data in the database. The flushing transaction must see whether any other transaction has committed a change to the entity in the intervening time since this transaction read it in and changed it. If a change occurred, it means that the flushing transaction has data that does not include those changes and should not write its own changes to the database lest it overwrite the changes from the intervening transaction. At this stage, it must roll back the transaction and throw a special exception called `OptimisticLockException`.

## Transactions & Locking

1. Change tx isolation level --to REPEATABLE\_READ

OR

2 Use "select for update" Query

Both of above approach applies a write lock

Better approach --Optimistic Locking

JPA 2 supports both optimistic locking and pessimistic locking. Locking is essential to avoid update collisions resulting from simultaneous updates to the same data by two concurrent users. Locking in JPA is always at the database object level, i.e. each database object is locked separately.

Optimistic locking is applied on transaction commit. Any database object that has to be updated or deleted is checked. An exception is thrown if it is found out that an update is being performed on an old version of a database object, for which another update has already been committed by another transaction.

Optimistic locking should be the first choice for most applications, since compared to pessimistic locking it is easier to use and more efficient.

In the rare cases in which update collision must be revealed earlier (before transaction commit) pessimistic locking can be used. When using pessimistic locking, database objects are locked during the transaction and lock conflicts, if they happen, are detected earlier.

Optimistic Locking

Add @Version annotated property in hibernate POJO.(data type Integer)

The initial version of a new entity object (when it is stored in the database for the first time) is 1. In every transaction in which an entity object is modified its version number is automatically increased by one.

During commit, hibernate checks every database object that has to be updated or deleted, and compares the version number of that object in the database to the version number of the in-memory object being updated. The transaction fails and an OptimisticLockException is thrown if the version numbers do not match, indicating that the object has been modified by another user (using another transaction) since it was retrieved by the current updater.

Pessimistic Locking

The main supported pessimistic lock modes are:

PESSIMISTIC\_READ - which represents a shared lock.

PESSIMISTIC\_WRITE - which represents an exclusive lock.

Setting a Pessimistic Lock

An entity object can be locked explicitly by the lock method:

org.hibernate.Session API

void lock(Object object, LockMode lockMode)

Obtain the specified lock level upon the given object. This may be used to perform a version check (LockMode.READ) or to upgrade to a pessimistic lock (LockMode.PESSIMISTIC\_WRITE)

eg :

```
sf.getCurrentSession().lock(employee, LockMode.PESSIMISTIC_WRITE);
```

## Design patterns

There are common problems faced by programmers all over.

Not a domain specific problem --asso with banking / insurance or manufacturing or pharma etc.

BUT if its a generic problem(eg : writing a web app , separating cross cutting concerns ) --then there are standard solutions to these common problems -- these are nothing but the design patterns---best practises.

Design Patterns -- Started with Gang Of Four in 1994 --but being modified/enhanced since then.

### Summary was

1. Always prefer composition over inheritance
2. Always code for i/f & not imple.

In a nut shell

Good programming practises --to be followed by prog community.

Should u know about all of them ? -- 100 %

Should u apply all of them -- NO ---nobody does that--you should only use those patterns which your appln demands.

### They are categorised into 3 parts

1. Creational design patterns -- best practises for object creation

eg : singleton , factory , builder

2. Structural -- best practises for composition of objects to create a larger application

eg : adapter

3. Behavioural -- best practises for communication between the objects (w/o any composition)

You can think of additional category as

4. J2EE design patterns -- MVC , Front Controller , DAO , DTO

Pattern examples

#### 1. singleton

Typically all spring beans , used are singleton.

#### 2. Factory

eg : o.s.bean.factory.BeanFactory <----- ApplicationContext <----- ClassPathXmlApplicationContext

In above case , SC is following factory pattern --provider of the rdymade spring beans (via ctx.getBean method)

Another eg : Hibernate's session factory

#### 2.5 Builder --

Use case --When there are too many parameters needed to be passed to a class constructor, instead use builder pattern.

### Structural Patterns

3. Adapter design pattern is one of the structural design pattern and its used so that two unrelated interfaces can work together. The object that joins these unrelated interface is called an Adapter.

eg : Mobile charger works as an adapter between mobile charging socket and the wall socket.

1. We will have two classes – Volt (to measure voltage) and Socket (producing constant voltage of 230V).

2. Now build an adapter that can produce 3 volts, 12 volts and default 230 volts. So create an adapter interface with these methods.

### **3. Object adapter implementation --based on composition**

#### **4. A tester**

-----

### **4. Facade Design Pattern -- structural design pattern**

Suppose we have an application with set of interfaces to use MySql/Oracle database and to generate different types of reports, such as HTML report, PDF report etc.

So we will have different set of interfaces to work with different types of database. Now a client application can use these interfaces to get the required database connection and generate reports.

But when the complexity increases or the interface behavior names are confusing, client application will find it difficult to manage it.

So we can apply Facade design pattern here and provide a wrapper interface on top of the existing interface to help client application.

eg :

1. Two helper interfaces, -- namely MySqlHelper and OracleHelper.

2. Facade pattern interface

3. Tester

-----

### **Proxy Design pattern is one of the Structural design pattern**

Provide a surrogate(substitute) or placeholder for another object to control access to it.

eg : AOP proxies.

Refer to example of tx manager in spring framework

eclipse project --test\_aop\_simple

-----

### **Behavioral Design patterns**

1. Template Method is a behavioral design pattern.

Template Method design pattern is used to create a method stub and deferring some of the steps of implementation to the subclasses.

eg : Spring example -- RestTemplate

-----

### **2. A Command pattern is an object behavioral pattern**

Allows us to achieve complete decoupling between the sender and the receiver.

A sender -- is an object that invokes an operation

A receiver is an object that receives the request to execute a certain operation.

With decoupling, the sender has no prior knowledge of the Receiver's interface.

Here request = command that is to be executed.

The Command pattern also allows us to vary when and how a request is fulfilled. Thus : a Command pattern provides us flexibility as well as extensibility.

eg :

1. 2 electrical instruments to operate --fan & light bulb
2. Command i/f
3. Command imple classes
4. Switch -- invokes command on the receiver object
5. tester

## SOAP vs REST web services

SOAP stands for simple object access protocol

REST stands for REpresentational State Transfer

### Protocol vs Architectural style

SOAP is a standard protocol to create web services

Rest is architectural style to create web services.

### Contract

Client and Server are bound with WSDL contract in SOAP

There is no contract between client and Server in REST

### Format Support

SOAP supports only XML format

REST web services supports XML, json and plain text etc.

### Maintainability

SOAP web services are hard to maintain as if we do any changes in WSDL , we need to create client stub again

REST web services are generally easy to maintain.

### Service interfaces vs URI

SOAP uses Service interfaces to expose business logic

Rest uses URI to expose business logic

### Security

SOAP has its own security : WS-security

Rest inherits its security from underlying transport layer.

### **Bandwidth**

SOAP requires more bandwidth and resources as it uses XML messages to exchange information

REST requires less bandwidth and resources. It can use JSON also.

### **Learning curve**

SOAP web services are hard to learn as you need to understand WSDL , client stub.

REST web services are easy to understand as you need to annotate plain java class with JAX-RS annotations to use various HTTP methods .

## **Spring's method-level dependency injection support, via the @Lookup annotation.**

### **2. Why @Lookup?**

A method annotated with @Lookup tells Spring to return an instance of the method's return type when we invoke it.

Essentially, Spring will override our annotated method and use our method's return type and parameters as arguments to BeanFactory#getBean.

#### **@Lookup is useful for:**

Injecting a prototype-scoped bean into a singleton bean (similar to Provider)

Injecting dependencies procedurally

Note also that @Lookup is the Java equivalent of the XML element lookup-method.

### **3. Using @Lookup**

#### **3.1. Injecting prototype-scoped Bean Into a Singleton Bean**

If we happen to decide to have a prototype Spring bean, then we are almost immediately faced with the problem of how will our singleton Spring beans access these prototype Spring beans?

Now, Provider is certainly one way, though @Lookup is more versatile in some respects.

First, let's create a prototype bean that we will later inject into a singleton bean:

#### **@Component**

#### **@Scope("prototype")**

```
public class SchoolNotification {  
    // ... prototype-scoped state  
}
```

And if we create a singleton bean that uses @Lookup:

#### **@Component**

```
public class StudentServices {  
    // ... member variables, etc.  
  
    @Lookup  
    public SchoolNotification getNotification() {
```

```

        return null;
    }

    // ... getters and setters
}

```

Using `@Lookup`, we can get an instance of `SchoolNotification` through our singleton bean:

### **@Test**

```

public void whenLookupMethodCalled_thenNewInstanceReturned() {
    // ... initialize context

    StudentServices first = this.context.getBean(StudentServices.class);
    StudentServices second = this.context.getBean(StudentServices.class);

    assertEquals(first, second);
    assertEquals(first.getNotification(), second.getNotification());
}

```

Note that in `StudentServices`, we left the `getNotification` method as a stub.

This is because Spring overrides the method with a call to `beanFactory.getBean(StudentNotification.class)`, so we can leave it empty.

### **3.2. Injecting Dependencies Procedurally**

Still more powerful, though, is that `@Lookup` allows us to inject a dependency procedurally, something that we cannot do with `Provider`.

Let's enhance `StudentNotification` with some state:

```

@Component
@Scope("prototype")
public class SchoolNotification {
    @Autowired Grader grader;

    private String name;
    private Collection<Integer> marks;

    public SchoolNotification(String name) {
        // ... set fields
    }

    // ... getters and setters

    public String addMark(Integer mark) {
        this.marks.add(mark);
    }
}

```

```

        return this.grader.grade(this.marks);
    }
}

```

Now, it is dependent on some Spring context and also additional context that we will provide procedurally.

## AD

We can then add a method to StudentServices that takes student data and persists it:

```

public abstract class StudentServices {
    private Map<String, SchoolNotification> notes = new HashMap<>();

    @Lookup
    protected abstract SchoolNotification getNotification(String name);

    public String appendMark(String name, Integer mark) {
        SchoolNotification notification
            = notes.computeIfAbsent(name, exists -> getNotification(name));
        return notification.addMark(mark);
    }
}

```

At runtime, Spring will implement the method in the same way, with a couple of additional tricks. First, note that it can call a complex constructor as well as inject other Spring beans, allowing us to treat SchoolNotification a bit more like a Spring-aware method.

It does this by implementing getSchoolNotification with a call to beanFactory.getBean(SchoolNotification.class, name).

Second, we can sometimes make the @Lookup-annotated method abstract, like the above example.

Using abstract is a bit nicer-looking than a stub, but we can only use it when we don't component-scan or @Bean-manage the surrounding bean:

## @Test

```

public void whenAbstractGetterMethodInjects_thenNewInstanceReturned() {
    // ... initialize context

    StudentServices services = context.getBean(StudentServices.class);

    assertEquals("PASS", services.appendMark("Alex", 89));
    assertEquals("FAIL", services.appendMark("Bethany", 78));
    assertEquals("PASS", services.appendMark("Claire", 96));
}

```

With this setup, we can add Spring dependencies as well as method dependencies to SchoolNotification.

## 4. Limitations

Despite @Lookup's versatility, there are a few notable limitations:

@Lookup-annotated methods, like getNotification, must be concrete when the surrounding class, like Student, is component-scanned. This is because component scanning skips abstract beans.

@Lookup-annotated methods won't work at all when the surrounding class is @Bean-managed.

In those circumstances, if we need to inject a prototype bean into a singleton, we can look to Provider as an alternative.

## 5. Conclusion

In this quick article, we learned how and when to use Spring's @Lookup annotation, including how to use it to inject prototype-scoped beans into singleton beans and how to use it to inject dependencies procedurally.

## Spring Security Flow

**1. Each incoming request passes through security filter chain for authentication and authorization process.**

eg : UsernamePasswordAuthenticationFilter.

OR If the incoming request contains the authorization header "Basic base-64 encoded credentials" , this request goes through the chains of filters until it reaches BasicAuthenticationFilter.

### What is the overall job of this filter?

Processes a HTTP request's Basic authorization header, putting the result into the SecurityContextHolder

**2. AuthenticationToken is created based on User Credentials** For the user login, once the authentication request reaches the authentication filter, it will extract the username and password from the request payload. Spring security will create an Authentication object based on the username and password.

UsernamePasswordAuthenticationToken authentication

```
= new UsernamePasswordAuthenticationToken(username, password);
```

**3. Authentication filter delegates the request to Authentication Manager.**

Authentication Manager is the core for the Spring security authentication process.

```
public interface AuthenticationManager {
```

```
    Authentication authenticate(Authentication authentication) throws AuthenticationException;
```

```
}
```

**4. This interface is implemented by ProviderManager class.**

ProviderManager has no idea about which auth provider will support the current authentication.

So it iterates through the list of AuthenticationProviders n calls supports() method

## 5. AuthenticationProviders

The AuthenticationProvider are responsible to process the request and perform a specific authentication. It provides a mechanism for getting the user details with which we can perform authentication.

```
public interface AuthenticationProvider {
```

```
    Authentication authenticate(Authentication authentication) throws AuthenticationException;
```

```
    boolean supports(Class<?> authentication);
```

```
}
```

eg of implementation classes :

DaoAuthenticationProvider.

RememberMeAuthenticationProvider

LdapAuthenticationProvider

You can also supply the custom authentication provider.

## **6. Typically used AuthenticationProvider is : DaoAuthenticationProvider**

It depends upon

1. UserDetailsService

2. PasswordEncoder

### **6.1 Spring Security UserDetailsService**

DaoAuthenticationProvider needs UserDetailsService to get the user details stored in the database by username

package org.springframework.security.core.userdetails;

```
public interface UserDetailsService {
```

```
    UserDetails loadUserByUsername(String userName/email) throws UsernameNotFoundException;
```

```
}
```

6.2 In case of in memory based user details , UserDetailsService is implemented by InMemoryUserDetailsManager

So you can configure this as a spring bean (@Bean) in your spring sec configuration file

BUT in practical scenario , we use DB based credentials.

In that case , we will implement UserDetailsService to provide User Details

## **7. Authentication and Authentication Exception**

During the authentication process, if the user authentication is successful, AuthenticationProvider will send a fully initialized Authentication object back.

For failed authentication, AuthenticationException will be thrown, filter will send the response to the client HTTP 401

For failed authorization , filter will send the response to the client HTTP 403 (forbidden)

Otherwise a fully populated authentication object carries the following details:

User credentials.

List of granted authorities (for authorization).

Authentication flag : set to true

## **8. Setting Authentication SecurityContext**

The last step in the successful authentication is setting up the authentication object in the SecurityContext. It wraps the SecurityContext in the SecurityContextHolder.

## **9. The request is then delegated further to DispatcherServlet n continues with the usual flow.**

=====

## JWT Details

### Session-based Authentication & Token-based Authentication

For using any website, mobile app or desktop app , you need to create an account, then use it to login for accessing features of the app : Authentication.

So, how to authenticate a user?

Simple method used : Session-based Authentication.

(refer : session-based-authentication.png)

As per the diag , when a user logs into a website, the Server will create a new Session for that user and store it (in Memory or Database). Server also returns a SessionId for the Client to save it in Browser Cookie.

The Session on Server has an expiration time. After that time, this Session has expired and the user must re-login to create another Session.

If the user has logged in and the Session has not expired yet, the Cookie (including SessionId) always goes with all HTTP Request to Server. Server will compare this SessionId with stored Session to authenticate and return corresponding Response.

If it's been working fine , then why do we need Token-based Authentication?

The answer is we don't have the only consumer as a browser (end user : client a person), we may have different consumers of our services here.

So if you have a website which works well with thin client(browser client) & want to implement system for Mobile (Native Apps) and use the same Web app. You can't authenticate users who use Native App using Session-based Authentication because these native apps don't support cookies. Or if you are implementing a REST server , every REST request HAS TO be stateless , meaning you can't think of maintaining a Http Session in the back end. On the other hand , you can't expect your front end app , to send you the credentials (username / password) along with every request.

That's why Token-based Authentication was born.

With this method, the user login state is encoded into a JSON Web Token (JWT) by the Server and sent to the Client after successful authentication.

Instead of creating a Session, the Server generates a JWT from user login data and sends it to the Client. The Client saves the JWT and then onwards sends this JWT along with every request , typically in a header to the server. The Server will validate the JWT and return the Response.

(Giving access to the secured resources)

For storing JWT on Client side, it depends on the platform you use:

Browser: Local Storage

IOS: Keychain

Android: SharedPreferences

### 3 important parts of a JWT:

**Header**

**Payload**

**Signature**

## 1. Header

The Header answers the question: How will we calculate JWT?

It's a JSON object

eg :

```
{  
  "typ": "JWT",  
  "alg": "HS512"  
}
```

– typ is 'type', indicates that Token type here is JWT.

– alg stands for 'algorithm' which is a hash algorithm for generating Token signature. Here HS256 is HMAC-SHA256/512 – the algorithm which uses Secret Key.

## 2. Payload

The Payload helps us to answer: What do we want to store in JWT?

This is a payload sample:

```
{  
  "userId": "abcd123456",  
  "username": "rama",  
  "email": "rama@gmail.com",  
  // standard fields  
  "iss": "Issuer at developers.com",  
  "iat": 1570238918,  
  "exp": 1570238992  
}
```

In the JSON object above, we store 3 user fields: userId, username, email. You can save any field you want.

We also have some Standard Fields. They are optional.

iss (Issuer): who issues the JWT

iat (Issued at): time the JWT was issued at

exp (Expiration Time): JWT expiration time

## 3. Signature

This part is where we use the Hash Algorithm : HMAC-SHA256

Look at the code for getting the Signature below:

In Java code :

java.util.Base64 class offers encoding n decoding.

Base64 is a binary-to-text encoding scheme. It represents binary data in a printable ASCII string format by translating it into a radix-64 representation.

The basic encoder keeps things simple and encodes the input as is, without any line separation.

The output is mapped to a set of characters in A-Za-z0-9+/ character set, and the decoder rejects any character outside of this set.

eg : In Java

```
String originalInput = "test input";
```

```
String encodedString = Base64.getEncoder().encodeToString(originalInput.getBytes());
```

In Javascript

```
const data = Base64UrlEncode(header) + '.' + Base64UrlEncode(payload);
```

```
const hashedData = Hash(data, secret);
```

```
const signature = Base64UrlEncode(hashedData);
```

– First, it encodes Header and Payload, join them with a dot .

- Next, makes a hash of the data using Hash algorithm (defined at Header) with a secret string.

– Finally, encodes the hashing result to get Signature.

After having Header, Payload, Signature, combines them into JWT standard structure: header.payload.signature.

### **Does JWT secures our data ?**

JWT does NOT secure your data

JWT does not hide, obscure, secure data at all. You can see that the process of generating JWT (Header, Payload, Signature) only encode & hash data, not encrypt data.

The purpose of JWT is to prove that the data is generated by an authentic source.

So, what if there is a Man-in-the-middle attack that can get JWT, then decode user information?

Yes, that is possible, so always make sure that your application has the HTTPS encryption.

### **How Server validates JWT from Client ?**

For creating a JWT , we use a Secret string to create Signature. This Secret string is unique for every Application and must be stored securely in the server side.

When receiving JWT from Client, the Server get the Signature, verify that the Signature is correctly hashed by the same algorithm and Secret string as above. If it matches the Server's signature, the JWT is valid.

---

## **Aspect Oriented Programming(AOP)**

### **WHY ?**

Separating cross-cutting concerns(=repeatative tasks) from the business logic/ request handling /persistence

### **IMPORTANT**

The key unit of modularity in OOP is the class, whereas in AOP the unit of modularity is the aspect. Dependency Injection helps you decouple your application objects from each other(eg : Controller separate from Service or DAO layers) and AOP helps you decouple cross-cutting concerns from the objects that they affect.(eg : transactional code or security related code can be kept separate from B.L or exception handling code from request handling method)

eg of ready made aspects provided by SC : tx management, security , exc handling

eg scenario --

Think of a situation Your manager tells you to do your normal development work(eg - write stock trading appln) + write down everything you do and how long it takes you.

A better situation would be you do your normal work, but another person observes what you're doing and records it and measures how long it took.

Even better would be if you were totally unaware of that other person and that other person was able to also observe and record, not just yours but any other people's work and time.

That's separation of responsibilities. --- This is what Spring offers you through AOP.

It is NOT an alternative to OOP BUT it complements OOP.

The key unit of modularity in OOP is the class, whereas in AOP the unit of modularity is the aspect.

Aspects enable the modularization of concerns.(concern=task/responsibility) such as transaction management,logging,security --- that cut across multiple types and objects. (Such concerns are often termed crosscutting concerns in AOP jargon)

Enables the modularization of cross cutting concerns(=task)

Eg : Logging,Security,Transaction management, Exception Handling

Similar in functionality to ---In EJB framework -- EJBObject

Struts 2 -- interceptors

Servlet -- filters.

RMI -- stubs

Hibernate --- proxy (hib framework -- lazy --- load or any--many associations --rets typically un-initiated proxy/proxies)

AOP with Spring Framework

One of the key components of Spring Framework is the Aspect oriented programming (AOP) framework.

Like DI, AOP supports loose coupling of application objects.

The functionalities that span multiple points of an application are called cross-cutting concerns.

With AOP, applicationwide concerns(common concerns-responsibilities or cross-cutting concerns like eg - declarative transactions, security,logging,monitoring,auditing,exception handling....)

are decoupled from the objects to which they are applied.

It's better for application objects(service layer/controller/rest controller/DAO) to focus on the business domain problems that they are designed for and leave certain ASPECTS to be handled by someone else.

Job of AOP framework is --- Separating these cross-cutting concerns(repeatative tasks) from the core business logic

AOP is like triggers in programming languages such as Perl, .NET.

Spring AOP module provides interceptors to intercept an application, for example, when a method is executed, you can add extra functionality before or after the method execution.

## Key Terms of AOP

**1.Advice :** Action(=cross cutting concern) to take either before/after or around the method (req handling logic, OR B.L OR data access logic) execution. eg: transactional logic(begin tx,commit,rollback,session closing)

Advice describes WHAT is to be done & WHEN it's to be done.

eg : logging. It is sure that each object will be using the logging framework to log the event happenings , by calling the log methods. So, each object will have its own code for logging. i.e the logging functionality requirement is spread across multiple objects (call this as Cross cutting Concern, since it cuts across multiple objects). Wont it be nice to have some mechanism to automatically execute the logging code, before executing the methods of several objects?

**2. Join Point** : Place in application WHERE advice should be applied.(i.e which B.L methods should be advised)

(Spring AOP, supports only method execution join point )

**3. Pointcut** : Collection of join points.

It is the expression used to define when a call to a method should be intercepted.

eg :

```
@Pointcut("execution (Vendor com.app.bank.*.*(double))")
```

```
advice logic{....}
```

**4. Advisor Group** of Advice and Pointcut into a single unit.

**5. Aspect** : class representing advisor(advice logic + point cut definition)-- @Aspect -- class level annotation.

**6. Target** : Application Object containing Core domain logic.(To which advice gets applied at specified join points) -- supplied by Prog

**7. Proxy** : Object created after applying advice to the target object(created by SC dynamically by implementing typically service layer i/f) ---consists of cross cutting concern(repeatative jobs , eg : tx management,security, exc handling)

**8.Weaving** -- meshing(integration) cross cutting concern around B.L

(3 ways --- compile time, class loading time or spring supported --dynamic --method exec time or run time)

Examples of readymade aspects :

Transaction management & security.

Types of Advice --appear in Aspect class

@Before : This advice (cross cutting concern) logic gets Executed only before B.L method execution.

@AfterReturning Executes only after method returns in successful manner

@AfterThrowing - Executes only after method throws exception

@After -- Executes always after method execution(in case of success or failure)

@Around -- Most powerful, executes before & after.

### Regarding pointcuts

Sometimes we have to use same Pointcut expression at multiple places, we can create an empty method with @Pointcut annotation and then use it as expression in advices.

eg of PointCut annotation syntax

```
@Before("execution (* com.app.bank.*(..))")
```

```
@Pointcut("execution (* com.app.bank.*.*(double))")
```

```
// point cut expression
```

```
@Pointcut("execution (* com.app.service.*.add*(..))")
```

```
// point cut signature -- empty method .
```

```
public void test() {  
    }  
}
```

eg of Applying point cut

1. @Before(value = "test()")

```
public void logBefore(JoinPoint p) {.....}
```

2. @Pointcut("within(com.app.service.\*)")

```
public void allMethodsPointcut(){}  
@Before("allMethodsPointcut()")
```

```
public void allServiceMethodsAdvice(){...}
```

3.@Before("execution(public void com.app.model..set\*(\*))")

```
public void loggingAdvice(JoinPoint joinPoint){pre processing logic ....}
```

4. //Advice arguments, will be applied to bean methods with single String argument

```
@Before("args(name)")
```

```
public void logStringArguments(String name){....}
```

5. //Pointcut to execute on all the methods of classes in a package

```
@Pointcut("within(com.app.service.*)")
```

```
public void allMethodsPointcut(){}  
6.@Pointcut("execution(* com.core.app.service.*.*(..))") // expression
```

```
private void meth1() {} // signature
```

```
7.@Pointcut("execution(* com.app.core.Student.getName(..))")
```

```
private void test() {}  
-----
```

### Steps in AOP Implementation

1. Create core java project.

2. Add AOP jars to runtime classpath.

3. Add aop namespace to spring config xml.

4. To Enable the use of the @AspectJ style of Spring AOP & automatic proxy generation, add <aop:aspectj-autoproxy/>

5. Create Business object class. (using stereotype annotations)

6. Create Aspect class, annotated with @Aspect & @Component

7. Define one or more point cuts as per requirement

Eg of Point cut definition.

```
@PointCut("execution (* com.aop.service.Account.add*(..))")
```

```
public void test() {}
```

OR

```
@Before("execution (* com.aop.service.Account.add*(..))")
```

```
public void logIt()
```

```
{
```

```
    //logging advice code
```

```
}
```

Use such point cut to define suitable type of advice.

Test the application.

execution --- exec of B.L method

```
eg : @Before("execution (* com.app.bank.*.*(..))")
```

```
public void logIt() {...}
```

Above tell SC ---- to intercept ---ANY B.L method ---

having ANY ret type, from ANY class from pkg -- com.app.bank

having ANY args

Before its execution.

Access to the current JoinPoint

Any advice method may declare as its first parameter, a parameter of type `org.aspectj.lang.JoinPoint` (In around advice this is replaced by `ProceedingJoinPoint`, which is a subclass of `JoinPoint`.)

The `org.aspectj.lang.JoinPoint` interface methods

1. `Object[] getArgs()` -- returns the method arguments.
2. `Object getThis()` --returns the proxy object
3. `Object getTarget()` --returns the target object
4. `Signature getSignature()` -- returns a description of the method that is being advised
5. `String toString()` -- description of the method being advised

=====

## Spring Boot Image handling : project objective

**Objective** : Image upload n download --from server side folder

### Use multipart request

An HTTP multipart request is an HTTP request that HTTP clients construct to send files and data over to an HTTP Server. It is commonly used by browsers and HTTP clients to upload files to the server.

`org.springframework.web.multipart.MultipartFile` => A representation of an uploaded file received in a multipart request.

A representation of an uploaded file received in a multipart request.

The file contents are either stored in memory or temporarily on disk. In either case, the user is responsible for copying file contents to a session-level or persistent store as and if desired. The temporary storage will be cleared at the end of request processing.

## Steps

0. Add a property in Entity n DTO , to store the image path

eg : private String imagePath;

1. To upload a file to a server side folder add the property (any property name)

file.upload.location=images

(This will be a folder relative to current project)

#limiting max file size for upload

spring.servlet.multipart.max-file-size=10MB

spring.servlet.multipart.max-request-size=15MB

2. File upload

In Application class , create "images" folder , if not present

2.1 Application class implements CommandLineRunner i/f n implement "run" method

It will run exactly once @ spring boot app launching time

OR

Create ImageHandlingService : as singleton n eager service bean

n create "images" folder , if not present

2.2 Create a rest controller

Other emp details are already stored in DB. image path has to be stored in the table n image should be uploaded in the "images" folder.

i/p : emp id : path var

MultipartFile : request param

resp : Emp DTO (with image file name set) or Simply ApiResponse

eg : Method : POST

http://localhost:8080/api/employees/{empId}/images

2.3 Image Handling Service

1. Create total path , using folder location ,file separator , image file's original file name

2. Copy multipart file into server side folder structure.

API of java.nio.file.Files : helper class

public static long copy(InputStream in, Path target, CopyOption... options) throws IOException

Copies all bytes from an input stream to a file returning no of bytes copied.

3. Save image path in Emp entity (to trigger update query) --setter

4. Return Emp DTO to the caller.(service --> controller --> JSON representation to client)

5. Test it with postman

3. Serving images (download) from server side folder.

i/p : emp id

Steps

3.1 In Controller

Method = GET

Add "produces" in the annotation(@GetMapping): to include image MediaType : JPEG\_VALUE , GIF\_VALUE, PNG\_VALUE

3.2 In service layer

1. get emp details from emp id
2. Get complete path of the image

emp's getter

3. API of java.nio.file.Files : helper class

Method :

public static byte[] readAllBytes(Path path) throws IOException

Reads all the bytes from a file. The method ensures that the file is closed when all bytes have been read or an I/O error, or other runtime exception, is thrown.

For java.nio.file.Path : Paths.get(String first,String ... more)

4. return contents of image file(byte[]) to the controller

From controller , simply add it in ResponseEntity n send it to the clnt.

4. For react frontend , use : in <img> : src as

Method : GET

<http://localhost:8080/api/employees/{empId}/image>

=====